



APTC106:

TALLER DE DESARROLLO WEB Y MOVIL

CASO

FOODPLEASE

Propuesta de Aplicación CRUD Web y Móvil

Informe Unificado de Avance

Sumativas Semanas 6, 8 y 9

Alumnos: PRISCILA ARGANARAZ
CARLOS GONZALEZ VILLEGAS
DANIEL HUERTA SALAZAR
OSVALDO MONCADA PERALTA

Profesor: ERNESTO VIVANCO

Fecha: Mayo 17 de 2026

ÍNDICE

ÍNDICE.....	2
1. INTRODUCCIÓN	5
1.1 Motivación.....	5
1.2 Objetivos del proyecto.....	5
1.3 Objetivos del informe	5
1.4 Estructura del documento	6
2. PLANTEAMIENTO DE LA PROPUESTA.....	7
2.1 Análisis del caso FoodPlease	7
2.2 Problema a solucionar	8
2.3 Beneficiarios de la solución.....	8
2.4 Solución propuesta	8
3. PLAN UTILIZADO Y METODOLOGÍA	9
3.1 Plan de trabajo por fases	9
3.2 Metodología de desarrollo.....	9
3.3 Herramientas de colaboración	9
3.4 Criterios de decisión	10
4. ALTERNATIVAS CONSIDERADAS Y RACIONALIZACIÓN	11
4.1 Framework backend: Django vs. Flask vs. FastAPI vs. Node/Express.....	11
4.2 Base de datos: SQLite vs. PostgreSQL vs. MySQL vs. MongoDB	12
4.3 Patrón arquitectónico: MVC simple vs. Capas con Service y Repository vs. Hexagonal	13
4.4 Frontend: server-side rendering vs. SPA vs. PWA	13
4.5 Diseño UI: Bootstrap vs. Tailwind vs. CSS puro.....	13
4.6 Stack móvil: React Native vs. Flutter vs. nativo (Kotlin/Swift) vs. PWA.....	14
4.7 Repositorio y despliegue Cloud: GitHub vs. GitLab vs. Bitbucket; Render vs. Railway vs. PythonAnywhere vs. Heroku	14
5. TRÁNSITO HACIA EL RESULTADO	16
5.1 Iteración 1 — Semana 3: análisis y propuesta conceptual	16
5.2 Iteración 2 — Semana 6: del ejemplo Django al MVP en Flask.....	16
5.3 Iteración 3 — Semana 8: repositorio, README y empaquetado móvil	16
5.4 Iteración 4 — Semana 9: mockups, integración y despliegue Cloud	17

5.5 Dificultades encontradas y resoluciones	17
6. ARQUITECTURA DEL SISTEMA IMPLEMENTADA	18
6.1 Diagrama por capas.....	18
6.2 Application Factory Pattern	18
6.3 Modelo de datos del dominio	19
6.4 Tabla resumen de entidades.....	19
6.5 Distinción explícita entre back-end, front-end web y front-end móvil	20
7. MODULARIDAD DE LOS COMPONENTES	21
7.1 Definición operacional de modularidad	21
7.2 Modularidad horizontal (por dominio)	21
7.3 Modularidad vertical (por capas)	21
7.4 Patrón Repository en detalle	22
7.5 Service Layer en detalle.....	23
7.6 Modularidad en el frontend (plantillas)	23
7.7 Ventajas concretas de la modularidad para FoodPlease.....	23
7.8 Limitaciones honestas.....	24
8. FUNCIONAMIENTO MÍNIMO VIABLE (MVP)	25
8.1 Estado de los módulos del MVP	25
8.2 Coincidencia entre documentación y comportamiento real	25
8.3 Capturas de pantalla del MVP en ejecución.....	26
8.4 Datos seed para reproducibilidad.....	27
8.5 Cómo ejecutar el MVP localmente	28
9. REPOSITORIO Y REPRODUCIBILIDAD	29
9.1 URL y estructura del repositorio.....	29
9.2 Contenido del README.....	29
9.3 Primer commit y convenciones de versionado	30
9.4 Verificación de reproducibilidad	30
10. EMPAQUETADO DEL APLICATIVO MÓVIL	31
10.1 Estrategia A — App nativa empaquetada como APK (Android) e IPA (iOS).....	31
10.2 Estrategia B — Progressive Web App (PWA)	31
10.3 Estrategia C — Cross-platform con React Native o Flutter (recomendación a futuro)	31
10.4 Comparativa y recomendación.....	32
11. DESPLIEGUE EN CLOUD.....	33
11.1 Decisión: Render	33

11.2 Plan de despliegue paso a paso	33
11.3 Consideraciones técnicas del despliegue.....	33
11.4 Plan de rollback	33
12. MOCKUPS Y PROPUESTA DE INTEGRACIÓN MÓVIL/WEB	35
12.1 Metodología prompt-to-app con Figma Make	35
12.2 Mockups de las cinco pantallas.....	35
12.3 Flujo de navegación	40
12.4 Estrategia de integración móvil/web	41
12.5 Tabla de endpoints REST	41
12.6 Consideraciones técnicas futuras	41
13. BUENAS PRÁCTICAS APLICADAS.....	43
13.1 Convenciones de código	43
13.2 Estructura de carpetas idiomática	43
13.3 Manejo de errores user-friendly	43
13.4 Configuración centralizada.....	43
13.5 Datos seed y reproducibilidad	43
13.6 Control de versiones disciplinado.....	43
14. CONCLUSIONES	44
14.1 Resumen de resultados	44
14.2 Capacidad crítica: qué salió bien y qué quedó débil.....	44
14.3 Logro de objetivos.....	44
14.4 Integración de la propuesta de mejora elaborada en la Semana 8.....	45
14.5 Integración de las dos estrategias de empaquetado del aplicativo móvil.....	45
14.6 Marco de acción futura.....	45
15. GLOSARIO Y REFERENCIAS BIBLIOGRÁFICAS.....	46
15.1 Glosario	46
15.2 Referencias bibliográficas	46
Anexo A — Evidencia del despliegue local del MVP	48
A.1 Instalación de dependencias y arranque del servidor Flask.....	48
A.2 Dashboard operativo en ejecución local	49
A.3 Módulos CRUD funcionando	50
A.4 Conclusión de la evidencia local.....	51

1. INTRODUCCIÓN

1.1 Motivación

La transformación digital del sector delivery ha experimentado un crecimiento sostenido en los últimos años, impulsado por nuevos hábitos de consumo, la digitalización de los comercios locales y la masificación del smartphone como canal de compra. En este escenario, los servicios de entrega de comida han pasado de operar con procesos manuales y aislados a requerir plataformas integradas que coordinen, en tiempo real, a clientes, comercios y repartidores.

FoodPlease es una plataforma de delivery que actualmente funciona con asignación manual de pedidos, sin seguimiento en tiempo real ni integración entre sus distintos actores. Esta forma de operar genera ineficiencias, errores de asignación, descoordinación y una experiencia limitada para el cliente final. El presente trabajo aborda esa brecha proponiendo una solución web y móvil integrada, sustentada en una arquitectura modular, escalable y reproducible.

La motivación de este informe no es solo entregar un MVP funcional, sino documentar con rigor el camino recorrido para llegar a la solución actual: las alternativas que evaluamos, los criterios con los que decidimos, las dificultades encontradas y la modularidad lograda. Esa narrativa de proceso, ausente en entregas previas, es la corrección central que este informe unificado introduce.

1.2 Objetivos del proyecto

Objetivo general:

- Diseñar e implementar una aplicación CRUD web —complementada con una propuesta móvil— para la gestión integral de pedidos, clientes, locales comerciales y repartidores de FoodPlease, aplicando los patrones MVC y Repository sobre un stack Python/Flask, con miras a un despliegue Cloud y a una arquitectura preparada para integrar una app móvil.

Objetivos específicos:

- Construir un CRUD web mínimamente viable que gestione las cuatro entidades del dominio (Clientes, Locales, Repartidores y Pedidos) con persistencia relacional.
- Aplicar el patrón Modelo-Vista-Controlador y complementarlo con una capa de Servicios y un patrón Repositorio que aislen reglas de negocio y acceso a datos.
- Documentar y justificar de manera explícita el plan utilizado, las alternativas técnicas consideradas, los criterios de descarte y la solución final adoptada.
- Asegurar la modularidad de los componentes para habilitar la futura integración con una aplicación móvil mediante API REST y el despliegue del sistema en un proveedor Cloud.

1.3 Objetivos del informe

Este informe unificado documenta el tránsito completo del proyecto desde el análisis del caso (Semana 3) hasta la entrega de la Semana 9, integrando los entregables intermedios de las Semanas 6 (Sumativa 2 — CRUD web) y 8 (Repository y empaquetado). Sus objetivos son cuatro:

- Explicitar el plan de trabajo, las decisiones tomadas y las alternativas descartadas, respondiendo de manera directa al feedback recibido en la Sumativa 2.
- Demostrar que el funcionamiento mínimo viable está alineado uno-a-uno con la documentación presentada y con el ejemplo CRUD utilizado como punto de partida.
- Detallar la modularidad de los componentes implementados, mostrando responsabilidades, interfaces y capacidad de extensión.
- Integrar la propuesta de mejora de la Semana 8 y la propuesta móvil de la Semana 9, cerrando el ciclo con conclusiones que verifican el logro de los objetivos planteados.

1.4 Estructura del documento

El informe se organiza en quince secciones agrupadas en tres bloques. El primer bloque (secciones 1 a 2) establece el marco conceptual y el planteamiento de la propuesta, reutilizando el análisis del caso elaborado en la Semana 3. El segundo bloque (secciones 3 a 7), que constituye el núcleo del rework respecto a entregas previas, expone el plan utilizado, las alternativas evaluadas con su racionalización, el tránsito hacia el resultado, la arquitectura implementada y la modularidad lograda. El tercer bloque (secciones 8 a 15) presenta los entregables operativos: el MVP web, el repositorio y reproducibilidad, las estrategias de empaquetado móvil, el despliegue Cloud, los mockups con su propuesta de integración, las buenas prácticas, las conclusiones integradoras y el glosario con la bibliografía consolidada.

2. PLANTEAMIENTO DE LA PROPUESTA

2.1 Análisis del caso FoodPlease

FoodPlease opera en un sector donde la inmediatez, la trazabilidad y la coordinación operativa son atributos críticos. Tres actores principales interactúan en cada pedido: el cliente, el local comercial que prepara la comida y el repartidor que la entrega. A ellos se suma un cuarto rol transversal, el administrador del sistema, responsable de supervisar el flujo operativo.

El diagnóstico del caso identificó las siguientes debilidades del sistema actual:

- Asignación manual de pedidos a repartidores, sin criterios automatizados de proximidad, disponibilidad o carga.
- Ausencia de seguimiento en tiempo real para clientes y administradores.
- Falta de integración entre los actores: cada uno trabaja con información parcial.
- Descoordinación operativa que se traduce en retrasos, errores de entrega y pérdida de pedidos.
- Sin optimización de rutas, sin notificaciones automáticas y sin reportes consolidados para la administración.

Estas debilidades, lejos de ser anecdóticas, se traducen en costos operacionales mayores, una experiencia del cliente deteriorada y una imposibilidad estructural de escalar la operación sin sumar personal a una gestión que sigue siendo artesanal.

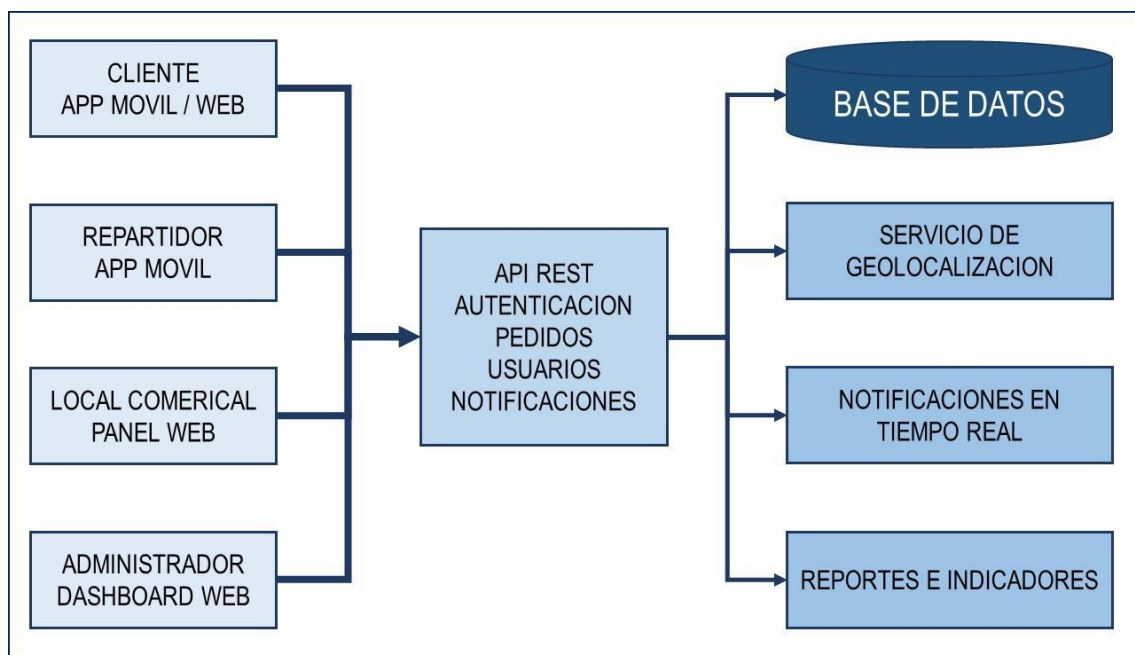


Figura 1. Arquitectura propuesta para FoodPlease (cliente, repartidor, local y administrador convergen en una API REST que coordina la base de datos y los servicios de geolocalización, notificaciones y reportes).

2.2 Problema a solucionar

El problema central es la ausencia de una solución tecnológica integrada que coordine, en tiempo real y de manera automatizada, a los actores del ecosistema FoodPlease. Esta carencia provoca procesos ineficientes, errores de asignación y un uso subóptimo de los recursos disponibles. Resolverlo requiere construir una plataforma digital con persistencia, lógica de negocio centralizada y vistas diferenciadas por rol, capaz de evolucionar hacia una arquitectura cliente-servidor con consumo móvil sobre una API.

2.3 Beneficiarios de la solución

La solución impacta de forma directa a cuatro tipos de actores:

- **Clientes:** realizan pedidos con menos fricción, conocen el estado en tiempo real y disponen de información consolidada del local y del repartidor asignado.
- **Locales comerciales:** reciben pedidos estructurados, gestionan su catálogo y su disponibilidad y acceden a un canal coordinado de despacho.
- **Repartidores:** reciben asignaciones claras, pueden marcar su disponibilidad y operan con un flujo estandarizado de aceptación-recolección-entrega.
- **Administradores:** supervisan métricas operacionales (pedidos del día, pedidos en ruta, repartidores activos), gestionan altas y bajas y toman decisiones basadas en datos.

2.4 Solución propuesta

La propuesta consiste en una plataforma digital integrada que automatice las operaciones críticas de FoodPlease. Las cinco mejoras concretas que la solución habilita son:

- Asignación automática de pedidos a partir de la disponibilidad y la ubicación del repartidor.
- Visualización del estado del pedido para el cliente, el local y el repartidor.
- Geolocalización para enrutamiento y trazabilidad de las entregas.
- Comunicación coordinada y notificaciones entre los actores.
- Información en tiempo real consolidada en un panel administrativo para la toma de decisiones.

Esta solución se materializa en una aplicación web —el panel administrativo y operativo, construido como CRUD modular en Flask— y una propuesta de aplicación móvil que consumirá la misma capa de servicios a través de una API REST. Las secciones siguientes detallan el plan, las decisiones y la implementación que dieron forma a esa visión.

3. PLAN UTILIZADO Y METODOLOGÍA

El profesor solicitó, en la retroalimentación de la Sumativa 2, que se explicitara qué plan se utilizó para llegar al resultado. Esta sección responde a esa solicitud documentando con rigor el plan de trabajo por fases, la metodología de desarrollo aplicada, las herramientas de colaboración y los criterios con los que se evaluaron las decisiones técnicas a lo largo del proyecto.

3.1 Plan de trabajo por fases

El proyecto se organizó en cuatro fases, cada una alineada con una entrega académica del curso. El plan privilegió la incrementalidad: cada fase consolidó lo construido en la anterior y agregó un nuevo nivel de madurez.

Fase	Entrega	Objetivo	Evidencia generada
Fase 1	Semana 3 — Sumativa 1	Análisis del caso y propuesta conceptual	Informe con problema, beneficiarios, solución y justificación técnica preliminar
Fase 2	Semana 6 — Sumativa 2	Construcción del MVP web (CRUD) con despliegue local	Aplicación Flask con las cuatro entidades, plantillas Bootstrap y datos seed
Fase 3	Semana 8 — Formativa 3	Repositorio público con README reproducible y reflexión sobre empaquetado móvil	Repositorio GitHub, README operativo y comparativa de estrategias de empaquetado
Fase 4	Semana 9 — Sumativa 3	Despliegue Cloud, mockups móviles y propuesta de integración móvil/web	Mockups de cinco pantallas, tabla de endpoints REST y plan de despliegue en Render

3.2 Metodología de desarrollo

Se adoptó una metodología iterativa e incremental con asignación por capas. La división del trabajo permitió que los integrantes del equipo avanzaran en paralelo sobre distintas partes del sistema (persistencia, lógica, rutas, plantillas, mockups) sin generar bloqueos. Las reuniones de sincronización fueron breves y orientadas a integrar lo construido por cada miembro antes de ejecutar un ciclo de pruebas conjunto.

Cada iteración terminó con una versión ejecutable del sistema —incluso si parcial— y con una actualización del informe. Esta disciplina evitó acumular deuda documental y permitió que el informe final consolidara material producido en cada fase, no recordado al cierre.

3.3 Herramientas de colaboración

- **Git y GitHub:** control de versiones, integración del trabajo del equipo, README como manual de reproducción.
- **Visual Studio Code:** IDE compartido con extensiones de Python y Flask.

- **Figma y Figma Make:** diseño de mockups móviles a partir de los prompts entregados por la cátedra.
- **Entornos virtuales (venv):** aislamiento de dependencias Python para asegurar reproducibilidad entre los sistemas operativos heterogéneos del equipo.
- **Canvas UNAB:** repositorio oficial de rúbricas, entregas y retroalimentación.

3.4 Criterios de decisión

Para evaluar cualquier alternativa técnica se aplicaron cinco criterios. Estos criterios actúan como una rúbrica interna y se reutilizan a lo largo de la sección 4 al comparar opciones:

- **1. Curva de aprendizaje:** qué tan rápido el equipo puede ser productivo con la tecnología.
- **2. Fit con el alcance docente:** cuán adecuada es la tecnología para un trabajo académico semestral, no para una operación productiva a gran escala.
- **3. Modularidad:** capacidad de descomponer el sistema en componentes independientes y reutilizables.
- **4. Escalabilidad futura:** posibilidad de migrar a infraestructura más robusta sin reescribir la solución.
- **5. Mantenibilidad:** claridad del código, documentación disponible y comunidad activa.

Cada decisión documentada en la sección siguiente se evalúa explícitamente contra estos cinco criterios antes de descartar alternativas y justificar la solución adoptada.

4. ALTERNATIVAS CONSIDERADAS Y RACIONALIZACIÓN

Esta sección es la respuesta directa al feedback de la Sumativa 2. Para cada decisión técnica clave se describen las alternativas evaluadas, los criterios aplicados, la racionalización de los descartes y la decisión final justificada. El espíritu de esta sección es exhibir el espacio de soluciones que el equipo recorrió antes de fijar el stack actual.

4.1 Framework backend: Django vs. Flask vs. FastAPI vs. Node/Express

Antes de detallar la comparativa es importante reconocer abiertamente un punto: la cátedra entregó, como material de referencia, un ejemplo CRUD construido en Django (carpeta simple-crud-django-master). Ese ejemplo se evaluó como primer candidato natural. Tras aplicar los criterios definidos en la sección 3.4, se decidió mantener el patrón MVC y la estructura modular del ejemplo, pero migrar el stack a Flask. Esta decisión —y su no-explicación en la entrega de la Semana 6— fue el motivo principal del castigo en Funcionamiento Mínimo Viable. A continuación se expone la racionalización completa.

Las cuatro alternativas evaluadas fueron:

- **Django:** framework full-stack maduro, batteries-included, con ORM, admin y autenticación integrada. Es el del ejemplo entregado por la cátedra.
- **Flask:** microframework minimalista, extensible mediante librerías, control granular del stack, comunidad madura.
- **FastAPI:** framework moderno orientado a APIs, asíncrono, tipado, foco en performance.
- **Node.js + Express:** plataforma JavaScript, microframework similar a Flask, fuerte para APIs en tiempo real.

Criterio	Django	Flask	FastAPI	Node/Express
Curva de aprendizaje	Alta (muchas convenciones)	Baja	Media (tipado, async)	Media
Fit con alcance docente	Sobrado para CRUD pequeño	Excelente	Excede el alcance	Cambia el lenguaje del curso
Modularidad	Apps + middleware	Blueprints + extensiones	Routers + Dependencies	Routers + middleware
Escalabilidad	Muy alta	Alta (con extensiones)	Muy alta para APIs	Alta
Mantenibilidad	Excelente (gran comunidad)	Excelente (gran comunidad)	Buena (más joven)	Buena

Tabla 1. Comparativa de frameworks backend evaluados.

Racionalización de descartes:

- **Django se descartó** porque sus convenciones full-stack (admin, ORM, formularios, autenticación) añaden complejidad innecesaria para un CRUD docente de cuatro entidades. El batteries-included es virtud en producción y carga conceptual en un trabajo

académico cuya prioridad pedagógica es comprender cada capa que se construye. Conservamos del ejemplo entregado el patrón MVC, la separación por módulos y la idea de plantillas reutilizables, pero migramos al stack que el equipo podía dominar en el plazo del curso.

- **FastAPI se descartó** porque su valor diferencial (tipado, asincronía, OpenAPI automático) está orientado a APIs de alto rendimiento. El proyecto requiere primero un CRUD web con plantillas server-side; la API REST queda como Fase 4 sobre la misma arquitectura.
- **Node/Express se descartó** porque obligaba a cambiar el lenguaje del curso (Python) y no aportaba ventajas sobre Flask para los objetivos planteados.

Decisión final: Flask.

Flask cumple en simultáneo la baja curva de aprendizaje y el alto fit con el alcance docente sin comprometer modularidad ni escalabilidad. La extensión Flask-SQLAlchemy aporta el ORM necesario, y los Blueprints habilitan la modularidad horizontal que demanda el caso (un módulo por entidad). La decisión es coherente además con el artículo de OpenWebinars referenciado por la cátedra, que destaca a Flask como alternativa con buena curva de aprendizaje, servidor de desarrollo integrado, buen manejo de rutas y compatibilidad con WSGI.

4.2 Base de datos: SQLite vs. PostgreSQL vs. MySQL vs. MongoDB

Alternativas evaluadas:

- **SQLite:** base de datos embebida, sin servidor, archivo único, integrada con Python.
- **PostgreSQL:** motor relacional avanzado, robusto, ideal para producción.
- **MySQL:** motor relacional popular, ampliamente soportado por hostings.
- **MongoDB:** base NoSQL documental, flexible en el esquema.

Criterio	SQLite	PostgreSQL	MySQL	MongoDB
Curva de aprendizaje	Mínima	Media	Media	Media-alta
Configuración inicial	Cero	Servidor + setup	Servidor + setup	Servidor + setup
Fit con alcance docente	Excelente	Excede	Excede	No relacional
Migración futura	Trivial a PostgreSQL	—	—	Reescribir queries
Integridad transaccional	ACID	ACID	ACID	Eventual (por defecto)

Tabla 2. Comparativa de motores de persistencia.

Racionalización y decisión:

Se descartó MongoDB por su modelo no relacional: el dominio de FoodPlease es naturalmente relacional (Pedido referencia a Cliente, Local y Repartidor), y forzar un modelo documental introduce complejidad sin beneficio. PostgreSQL y MySQL son superiores a SQLite en producción, pero exigen un servidor en ejecución, configuración de credenciales,

drivers adicionales y aumentan la fricción de reproducción del MVP en un sistema externo. Se decidió SQLite para el MVP, dado que la abstracción que ofrece SQLAlchemy permite migrar a PostgreSQL más adelante con un cambio de una línea en config.py.

4.3 Patrón arquitectónico: MVC simple vs. Capas con Service y Repository vs. Hexagonal

Tres alternativas evaluadas. MVC simple (la del ejemplo Django original) mantiene los controladores delgados y los modelos como entidades, sin capas intermedias. Hexagonal (puertos y adaptadores) ofrece desacople máximo entre dominio, casos de uso y adaptadores. Capas con Service y Repository es un punto intermedio.

Se descartó Hexagonal porque el costo conceptual y de código no se justifica para un CRUD docente; la inversión de dependencias y la separación dominio / aplicación / infraestructura son apropiadas para sistemas grandes con múltiples adaptadores. Se descartó MVC puro porque la práctica recomendada para Flask, y el feedback explícito del profesor sobre modularidad, sugieren introducir al menos un patrón Repository y una capa de Servicios para aislar reglas de negocio. La decisión fue: MVC con capa de Servicios y patrón Repository, materializado en la estructura Routes → Services → Repositories → Models. Esa misma estructura se detalla en las secciones 6 y 7.

4.4 Frontend: server-side rendering vs. SPA vs. PWA

Se evaluó construir el panel administrativo como Single Page Application en React/Vue, como Progressive Web App o mantener un enfoque clásico de server-side rendering con Jinja2 y Bootstrap. Las SPA aportan mejor experiencia pero introducen un build pipeline, dependencias adicionales y la necesidad de exponer una API desde el primer día. Las PWA son atractivas para el lado móvil, pero el panel administrativo no es el caso de uso natural. Se optó por renderizado server-side con Jinja2 + Bootstrap 5 porque minimiza la complejidad y deja la API REST como capa explícita de la Fase 4, cuando la app móvil la consuma.

4.5 Diseño UI: Bootstrap vs. Tailwind vs. CSS puro

Bootstrap fue la elección por dos razones: trae componentes listos (cards, tablas, formularios, alertas) coherentes con el estilo administrativo del proyecto, y la curva de aprendizaje del equipo es prácticamente nula. Tailwind es excelente para diseños personalizados, pero requiere un build pipeline. CSS puro implica reinventar componentes y dispersa el esfuerzo. Bootstrap 5 vía CDN aporta un baseline visual sólido en cero pasos de configuración.

4.6 Stack móvil: React Native vs. Flutter vs. nativo (Kotlin/Swift) vs. PWA

La propuesta móvil de la Semana 9 es —por instrucción de la cátedra— una propuesta de mockups y de integración, no una implementación nativa. Aun así se documentaron los caminos posibles para construirla:

Stack	Ventaja	Desventaja	Encaje en FoodPlease
React Native	Único código para iOS y Android, gran ecosistema	Performance algo inferior a nativo en animaciones complejas	Recomendado para MVP móvil
Flutter	Excelente rendimiento, una sola base de código	Lenguaje Dart, comunidad más joven	Buen candidato alternativo
Nativo (Kotlin/Swift)	Mejor performance, acceso pleno a APIs del sistema	Doble esfuerzo: dos bases de código separadas	Reservado para escalamiento futuro
PWA	Sin tiendas, instalable, basada en web	Limitaciones de hardware en iOS	Estrategia de empaquetado mínimo viable (sec. 10)

Tabla 3. Comparativa de stacks móviles para la integración futura.

Decisión: se recomienda iniciar con PWA para validar el flujo de usuario sin barreras de tiendas, y evolucionar hacia React Native cuando el producto valide tracción. Ambas opciones se desarrollan como estrategias formales de empaquetado en la sección 10.

4.7 Repositorio y despliegue Cloud: GitHub vs. GitLab vs. Bitbucket; Render vs. Railway vs. PythonAnywhere vs. Heroku

Para el repositorio se eligió GitHub por su madurez de ecosistema, integración con servicios Cloud y por ser estándar del curso. GitLab y Bitbucket ofrecen funcionalidad equivalente, pero la curva de aprendizaje y la cobertura de integraciones favorecieron a GitHub.

Para el despliegue Cloud se evaluaron cuatro alternativas:

Plataforma	Free tier	Encaje con Flask + SQLite	Curva	Decisión
Render	Sí (servicio web + 750 hrs/mes)	Excelente; soporta PostgreSQL gratis	Baja	Elegida
Railway	5 USD crédito/mes	Excelente	Baja	Backup
PythonAnywhere	Sí, limitado	Específico para Python	Baja	Descartada
Heroku	No gratuito desde 2022	Excelente histórico	Baja	Descartada

Tabla 4. Comparativa de plataformas de despliegue Cloud.

Decisión: Render. El free tier permite mantener el servicio activo durante el período de evaluación; el despliegue se realiza directo desde el repositorio GitHub con un comando definido en un archivo de configuración; y permite migrar a PostgreSQL gratuito sin cambiar el proveedor. El detalle del plan de despliegue se desarrolla en la sección 11.

5. TRÁNSITO HACIA EL RESULTADO

Esta sección recoge el segundo punto del feedback del profesor: explicitar qué se probó, qué se descartó y cómo se fue construyendo el resultado iteración a iteración. La narrativa sigue las cuatro fases del plan descrito en la sección 3 y cierra con las dificultades concretas encontradas en el camino.

5.1 Iteración 1 — Semana 3: análisis y propuesta conceptual

La primera iteración fue puramente conceptual. El equipo trabajó el caso entregado, identificó los tres actores principales más el administrador, diagnosticó las debilidades del sistema actual y elaboró una propuesta de solución integrada web + móvil con su justificación técnica. Esta entrega obtuvo puntaje máximo y estableció el lenguaje de dominio (glosario, beneficiarios, diagrama de arquitectura general) que se conserva en el presente informe.

5.2 Iteración 2 — Semana 6: del ejemplo Django al MVP en Flask

La segunda iteración fue la más decisiva y la más conflictiva. El equipo descargó y exploró el ejemplo CRUD en Django entregado por la cátedra. Al levantarlo, se detectaron tres fricciones que motivaron una reevaluación del stack: (a) la configuración del ORM Django requería más tiempo del disponible para que todos los integrantes pudieran trabajar en paralelo, (b) la cantidad de convenciones del framework excedía el alcance pedagógico esperado para un CRUD de cuatro entidades, y (c) la documentación que dominaba el equipo era principalmente sobre Flask. Tras aplicar los cinco criterios de la sección 3.4, se decidió migrar a Flask conservando el patrón MVC y la estructura modular del ejemplo.

Esta decisión no se explicitó adecuadamente en el informe de la Semana 6, lo que motivó el puntaje aceptable en Funcionamiento Mínimo Viable. El presente informe corrige esa omisión: la sección 4.1 documenta paso a paso la racionalización.

El MVP funcional en Flask se construyó en tres sub-iteraciones: primero, el esqueleto del proyecto (factory pattern, configuración, conexión SQLite); segundo, el módulo Clientes como módulo de referencia (modelo, repositorio, ruta, plantillas list/form), del cual se replicaron los patrones para Locales, Repartidores y Pedidos; tercero, el dashboard con métricas agregadas y los datos seed para que cualquier evaluador pudiera levantar el proyecto y ver contenido sin pasos manuales. Al final de esta iteración el sistema corría en <http://127.0.0.1:5000> con un panel de administración cuyas capturas se muestran en la sección 8.

5.3 Iteración 3 — Semana 8: repositorio, README y empaquetado móvil

La tercera iteración tuvo dos focos. El primero, técnico: publicar el repositorio en GitHub, definir una estructura de carpetas reproducible y escribir un README operativo con los pasos exactos para levantar el sistema en cualquier máquina (clonado, entorno virtual, instalación de dependencias, ejecución, acceso). Este README es el que la sección 9 documenta. El segundo foco, analítico: explorar formas de empaquetar la futura aplicación móvil para distribuirla. Se estudiaron dos estrategias —empaquetado nativo con APK/IPA y Progressive Web App— que se desarrollan en detalle en la sección 10.

5.4 Iteración 4 — Semana 9: mockups, integración y despliegue Cloud

La cuarta iteración completó la propuesta. Usando Figma Make sobre los prompts entregados por la cátedra, se produjeron cinco mockups de pantallas móviles (login, home, detalle de restaurante, carrito y seguimiento), descritos en la sección 12. En paralelo, se diseñó la API REST que la app móvil consumirá del backend Flask, consolidando una tabla de endpoints organizados por entidad. Finalmente, se definió el plan de despliegue Cloud en Render, documentado en la sección 11. Esta iteración cierra el ciclo del proyecto entregando una propuesta completa: panel administrativo web operativo, repositorio reproducible, mockups móviles, plan de integración y plan de despliegue.

5.5 Dificultades encontradas y resoluciones

El tránsito hacia el resultado tuvo obstáculos concretos. Documentarlos no es una concesión, sino una práctica de honestidad técnica que da cuenta del aprendizaje real:

- **Heterogeneidad de los sistemas operativos del equipo:** cada integrante trabajó sobre un entorno distinto (Windows, macOS, Linux). El uso de entornos virtuales venv y la fijación de versiones en requirements.txt absorbió esta variabilidad y garantizó que el código corriera idéntico en todas las máquinas.
- **Iteraciones del esquema de la base de datos:** la relación entre Pedido, Cliente, Local y Repartidor se ajustó varias veces. Inicialmente Repartidor se modeló como obligatorio en cada pedido; al revisar el flujo se entendió que un pedido puede crearse sin repartidor asignado y luego asignarse. El campo pasó a NULLABLE. Cada cambio se acompañó de la regeneración del archivo SQLite con db.create_all() y el script seed.
- **Conflictos al refactorizar plantillas Jinja2:** al introducir herencia con base.html, algunas plantillas hijas fallaban por bloques redefinidos. Se resolvió homologando todas las plantillas a la convención {% extends 'base.html' %} + {% block title %} + {% block content %}.
- **Divergencia entre versiones del informe de la Semana 9:** durante la elaboración se generaron dos documentos paralelos con URLs de repositorio distintas y secciones desbalanceadas. El presente informe consolida la versión definitiva y la URL única del repositorio (sección 9), eliminando la ambigüedad.

6. ARQUITECTURA DEL SISTEMA IMPLEMENTADA

La arquitectura del sistema combina tres patrones clásicos sobre la base MVC de Flask: el Application Factory para inicializar la aplicación, los Blueprints para modularizar por dominio y la combinación Service Layer + Repository Pattern para separar reglas de negocio del acceso a datos. El resultado es una arquitectura por capas en la que cada elemento tiene una responsabilidad única y comunicaciones explícitas con sus vecinos.

6.1 Diagrama por capas

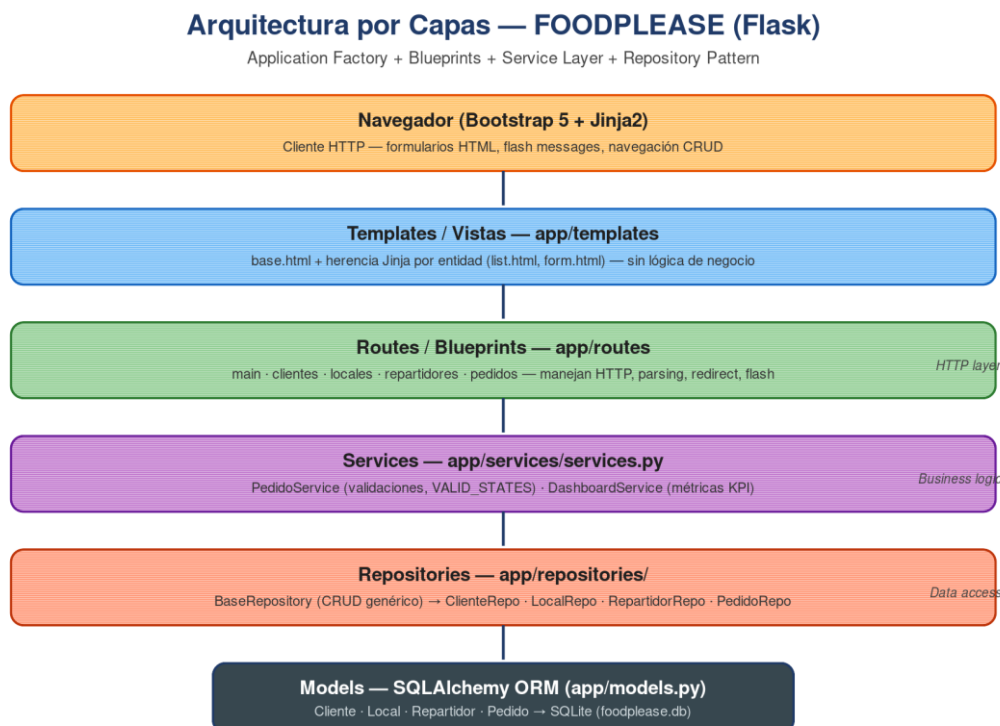


Figura 2. Arquitectura por capas implementada en FOODPLEASE. El flujo HTTP viaja del navegador a las plantillas, atraviesa las rutas, los servicios y los repositorios, y termina en los modelos SQLAlchemy persistidos en SQLite.

6.2 Application Factory Pattern

El módulo `app/__init__.py` expone una función `create_app()` que actúa como factory de la aplicación. Esta función instancia Flask, carga la configuración desde `config.py`, vincula la extensión SQLAlchemy mediante `db.init_app(app)`, crea las tablas con `db.create_all()` dentro del `app_context()` y registra los cinco Blueprints (`main`, `clientes`, `locales`, `repartidores` y `pedidos`). La adopción del factory permite que distintas configuraciones —desarrollo, pruebas, producción— produzcan instancias independientes de la app sin estados compartidos.

6.3 Modelo de datos del dominio

Cuatro entidades componen el dominio: Cliente, Local, Repartidor y Pedido. Las tres primeras son maestras y se mantienen con CRUD plano; la última es la entidad transaccional central, ya que referencia simultáneamente a las otras tres. Todas comparten una clave primaria entera autoincremental y un campo `created_at` de timestamp UTC.

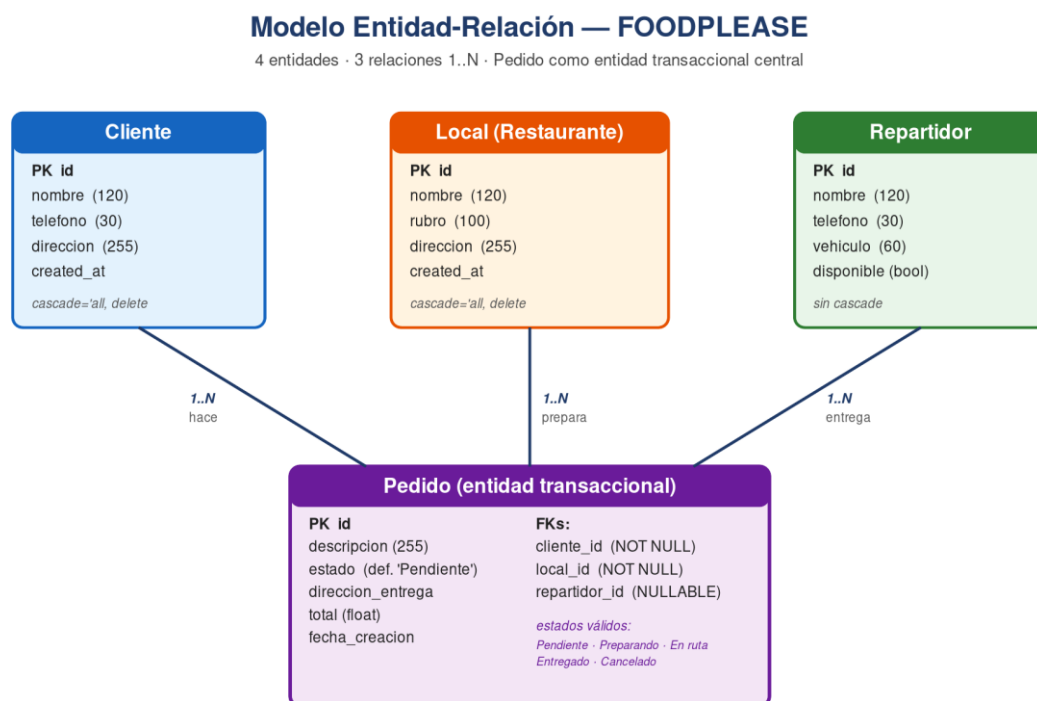


Figura 3. Modelo entidad-relación de FOODPLEASE. Cliente, Local y Repartidor tienen una relación 1..N con Pedido. El campo `repartidor_id` es NULLABLE para permitir pedidos sin repartidor asignado.

6.4 Tabla resumen de entidades

Modelo	Atributos principales	Relaciones	Observaciones
Cliente	id (PK), nombre, telefono, direccion, created_at	1..N con Pedido (cascade='all, delete')	Quien solicita el pedido
Local	id (PK), nombre, rubro, direccion, created_at	1..N con Pedido (cascade='all, delete')	Comercio que prepara
Repartidor	id (PK), nombre, telefono, vehiculo, disponible (bool), created_at	1..N con Pedido (sin cascade)	Recurso operacional
Pedido	id (PK), descripcion, estado, direccion_entrega, total, fecha_creacion	N..1 con Cliente, Local; N..1 con Repartidor (FK nullable)	Entidad transaccional central

Tabla 5. Resumen de las cuatro entidades del modelo de dominio.

El modelo permite que un Pedido se cree apenas el cliente lo solicita, sin necesidad de tener un repartidor asignado todavía. La asignación posterior actualiza el campo `repartidor_id` mediante la lógica de `PedidoService.update()`. Esta flexibilidad refleja la operación real del negocio.

6.5 Distinción explícita entre back-end, front-end web y front-end móvil

Una lectura precisa del diagrama por capas exige distinguir con claridad tres ámbitos que la solución FoodPlease aborda con distintos niveles de implementación. Confundirlos lleva a malinterpretar el alcance del trabajo, por lo que esta subsección los define de forma explícita.

- **Back-end:** es la totalidad del código Python que se ejecuta en el servidor — los Blueprints (`app/routes/`), los Servicios (`app/services/`), los Repositorios (`app/repositories/`) y los Modelos (`app/models.py`). Es el cerebro del sistema: aplica las reglas de negocio, gestiona la persistencia en SQLite y expone las URLs que cualquier cliente (navegador web hoy, aplicación móvil mañana) consume. El back-end está implementado, probado y documentado en este informe, y su evidencia de funcionamiento se presenta en la sección 8 y en el Anexo A.
- **Front-end web:** es el panel administrativo que renderiza el navegador. Está construido con plantillas Jinja2 (`base.html` + plantillas por entidad) estilizadas con Bootstrap 5 vía CDN. Es un frontend de renderizado del lado del servidor (`server-side rendering`): el back-end arma el HTML completo y lo entrega al navegador en cada request. No hay JavaScript reactivo ni una capa SPA, por elección explícita justificada en la sección 4.4. El front-end web está implementado y funcionando, y constituye la cara visible del MVP demostrado en las capturas del Anexo A.
- **Front-end móvil:** es la propuesta visual y funcional de la aplicación móvil para el cliente final, diseñada con Figma Make. En esta entrega se materializa exclusivamente como mockups (cinco pantallas) y como propuesta de integración con el back-end vía API REST. Esta decisión no es una omisión: la rúbrica de la Semana 9 indica textualmente que se deben realizar mockups "sin lógica asociada; solo primeras interacciones de navegación". El front-end móvil se documenta en la sección 12 del informe.

El punto clave es que el back-end ya está diseñado de forma agnóstica del transporte. Cuando la aplicación móvil real se implemente en una fase posterior (con React Native, Flutter o nativamente), ese front-end nuevo consumirá exactamente los mismos servicios y repositorios que hoy alimentan al panel web; la única adición será un Blueprint `/api/v1` que devuelva `jsonify(...)` en lugar de `render_template(...)`. Esa simetría es la consecuencia directa de la modularidad por capas expuesta en la sección siguiente.

7. MODULARIDAD DE LOS COMPONENTES

La modularidad fue el segundo eje crítico del feedback recibido. Esta sección la documenta de forma sistemática: define qué entendemos por módulo, describe la modularidad horizontal (por dominio) y la vertical (por capas), profundiza en los patrones Repository y Service Layer, explica la modularidad del frontend y cierra con las ventajas concretas y las limitaciones honestas detectadas.

7.1 Definición operacional de modularidad

En este proyecto se entiende por módulo un componente que cumple cuatro condiciones: tiene una responsabilidad única y bien delimitada; expone una interfaz clara hacia el resto del sistema; puede ser sustituido por otra implementación que cumpla la misma interfaz; y puede probarse de forma aislada. La modularidad es una propiedad emergente del sistema cuando todos sus componentes cumplen estas cuatro condiciones.

7.2 Modularidad horizontal (por dominio)

El sistema se descompone horizontalmente en cinco Blueprints: main (dashboard), clientes, locales, repartidores y pedidos. Cada Blueprint es un módulo completo: tiene su propio `url_prefix`, sus rutas, sus plantillas y su repositorio asociado. Agregar un módulo nuevo —por ejemplo Promociones— requiere crear un archivo en `app/routes/`, declarar una clase en `app/repositories/repositories.py`, agregar las plantillas correspondientes y registrar el Blueprint en el factory. La intervención sobre los módulos existentes es nula.

Blueprint	url_prefix	Endpoints	Capa intermedia
main_bp	/	GET / (dashboard con métricas)	DashboardService
clientes_bp	/clientes	GET /, GET/POST /nuevo, GET/POST /<id>/editar, POST /<id>/eliminar	Solo Repository
locales_bp	/locales	Estructura idéntica al CRUD anterior	Solo Repository
repartidores_bp	/repartidores	CRUD + manejo de flag disponible	Solo Repository
pedidos_bp	/pedidos	CRUD + selectores cliente/local/repartidor + validación de estado	PedidoService + 4 Repositories

Tabla 6. Modularidad horizontal: Blueprints por dominio.

7.3 Modularidad vertical (por capas)

El sistema se descompone verticalmente en cuatro capas con responsabilidades estrictas y reglas de comunicación explícitas:

- **Routes (HTTP):** parsing del request.form, manejo de flash messages, redirects y render de plantillas. Las rutas no tocan `db.session` ni ejecutan reglas de negocio.

- **Services (lógica de negocio):** validaciones, reglas de dominio, normalización de datos, métricas agregadas. Los servicios lanzan ValueError cuando la regla se incumple; las rutas atrapan estos errores y los muestran como flash danger.
- **Repositories (acceso a datos):** monopolizan el uso de SQLAlchemy. Toda operación CRUD pasa por un repositorio. Ningún Blueprint importa db directamente.
- **Models (persistencia):** definen las tablas, los atributos y las relaciones SQLAlchemy. No contienen reglas de negocio.

Esta separación tiene una consecuencia práctica importante: la lógica de negocio es agnóstica del transporte HTTP. Cuando la Fase 4 introduzca una API REST para la app móvil, ese nuevo Blueprint /api/v1 reutilizará exactamente los mismos servicios y repositorios. La diferencia con el panel web será únicamente que en lugar de render_template devolverá jsonify.

7.4 Patrón Repository en detalle

El patrón Repository se implementa en dos niveles. En app/repositories/base_repository.py se define una clase BaseRepository genérica que expone los métodos CRUD universales: get_all(), get_by_id(), create(), update() y delete(). El atributo de clase model se sobrescribe en las subclases.

```
class BaseRepository:
    model = None # subclasses override

    @classmethod
    def get_all(cls):
        return cls.model.query.order_by(cls.model.id.desc()).all()

    @classmethod
    def get_by_id(cls, id):
        return cls.model.query.get_or_404(id)

    @classmethod
    def create(cls, **kwargs):
        obj = cls.model(**kwargs)
        db.session.add(obj)
        db.session.commit()
        return obj
```

En app/repositories/repositories.py los repositorios concretos heredan de BaseRepository y declaran únicamente el modelo. Cuando un repositorio necesita una consulta específica de su dominio, la agrega como método de clase. Es el caso de RepartidorRepository, que añade get_available() para devolver únicamente los repartidores marcados como disponibles.

```
class ClienteRepository(BaseRepository):
    model = Cliente

class RepartidorRepository(BaseRepository):
    model = Repartidor
```

```

@classmethod
def get_available(cls):
    return
cls.model.query.filter_by(disponible=True).order_by(cls.model.nombre).all()

```

Las ventajas concretas de esta organización son tres: el principio DRY se respeta porque la lógica CRUD se escribe una sola vez en la base; la sustituibilidad queda garantizada porque cambiar el motor de persistencia o introducir un caché solo afecta a la capa Repository; y la testabilidad se mejora porque los servicios pueden ser probados con repositorios simulados sin levantar la base de datos.

7.5 Service Layer en detalle

El módulo `app/services/services.py` contiene dos clases. `DashboardService.build_metrics()` agrega las métricas del panel principal: cuenta entidades, agrupa pedidos por estado y devuelve un diccionario consumido por la plantilla `index.html`. `PedidoService` encapsula las reglas de negocio del módulo más complejo. Mantiene una constante de dominio `VALID_STATES` con los cinco estados permitidos (Pendiente, Preparando, En ruta, Entregado, Cancelado), que actúa como fuente única de verdad: la misma constante se consume tanto en `validate_payload()` como en el selector del formulario de pedidos.

Cuando el formulario de creación o edición de pedidos se envía, la ruta delega la validación completa al servicio. El servicio verifica campos obligatorios, convierte el total a flotante, exige `total > 0` y valida que el estado esté dentro de `VALID_STATES`. Si alguna regla se incumple, lanza un `ValueError` con un mensaje listo para mostrar al usuario. Solo si la validación pasa, el servicio delega la operación al repositorio. La ruta no toma decisiones de negocio: su única tarea es manejar HTTP.

7.6 Modularidad en el frontend (plantillas)

El frontend reproduce el mismo principio. La plantilla maestra `base.html` declara el layout global (navbar oscuro con identidad FoodPlease Admin Pro, contenedor de mensajes flash con categorías Bootstrap, y los bloques `title` y `content`). Por cada entidad existen dos plantillas: `list.html` para la tabla CRUD con acciones de edición y eliminación, y `form.html` que sirve simultáneamente para crear y editar —distingue ambos casos con un condicional Jinja `{% if cliente %}`. Toda la identidad visual se hereda; ninguna plantilla duplica estructura.

7.7 Ventajas concretas de la modularidad para FoodPlease

La arquitectura habilita cinco capacidades de extensión y mantenimiento con costo mínimo:

- Agregar un módulo nuevo (por ejemplo, Promociones) requiere un archivo en `routes/`, una clase en `repositories.py` y una línea en el `factory`. Cero impacto en los módulos existentes.
- Migrar la persistencia de SQLite a PostgreSQL: una sola línea en `config.py`. Los repositorios y servicios no cambian porque SQLAlchemy abstrae el driver.

- Exponer una API REST para la app móvil: un Blueprint /api/v1 que reutilice los mismos repositorios y servicios. La lógica de negocio es agnóstica del transporte HTML/JSON.
- Sustituir Bootstrap por otro framework de UI: editar exclusivamente base.html y los archivos list.html y form.html. La lógica de Python no se toca.
- Introducir autenticación con Flask-Login: agregar un módulo Usuario, un Blueprint auth y decoradores en las rutas. Los repositorios no se modifican.

7.8 Limitaciones honestas

Una sección de modularidad sería incompleta sin reconocer sus tensiones. El proyecto presenta tres oportunidades de mejora que se incluyen en el marco de acción futura:

- **Service Layer parcialmente consistente:** los módulos Clientes, Locales y Repartidores van directamente de la ruta al repositorio. Solo Pedidos y Dashboard usan un servicio. Una iteración futura puede agregar ClienteService, LocalService y RepartidorService aunque inicialmente solo deleguen, dejando la arquitectura simétrica.
- **Migraciones de esquema sin Alembic:** el proyecto usa db.create_all(), que no soporta evolución del esquema. Adoptar Flask-Migrate (Alembic) permitirá cambios incrementales en producción.
- **Validaciones del lado servidor en Pedidos:** el resto de los módulos confía en los atributos required de HTML. Adoptar Flask-WTF permitiría validaciones declarativas y protección CSRF en toda la app.

8. FUNCIONAMIENTO MÍNIMO VIABLE (MVP)

Esta sección documenta el funcionamiento mínimo viable de FoodPlease, el criterio donde la Sumativa 2 obtuvo puntaje aceptable. La estrategia de corrección consiste en mostrar de forma explícita la correspondencia uno-a-uno entre la documentación y el comportamiento real del sistema, así como en evidenciar cada módulo con su captura correspondiente.

8.1 Estado de los módulos del MVP

Módulo	Funcionalidades CRUD	Estado	Evidencia
Dashboard	Métricas KPI (entidades, pedidos por estado), últimos pedidos	Funcional	Figura 4
Cientes	Crear, listar, editar, eliminar — campos: nombre, teléfono, dirección	Funcional	Figura 5
Locales	Crear, listar, editar, eliminar — campos: nombre, rubro, dirección	Funcional	Figura 6
Repartidores	Crear, listar, editar, eliminar — campos: nombre, teléfono, vehículo, disponible (boolean)	Funcional	Figura 7
Pedidos	Crear, listar, editar, eliminar — selectores de cliente/local/repartidor, estados (Pendiente, Preparando, En ruta, Entregado, Cancelado), total con validación > 0	Funcional	Figura 8

Tabla 7. Estado de los cinco módulos del MVP.

8.2 Coincidencia entre documentación y comportamiento real

Este apartado cierra la brecha que el feedback del profesor explicitó. Cada función documentada tiene su captura asociada y su endpoint trazable en el código fuente:

- **Dashboard:** documentado como panel agregado con tarjetas KPI y tabla de últimos cinco pedidos. Comportamiento real: la ruta GET / invoca DashboardService.build_metrics() y renderiza index.html. La Figura 4 muestra el panel ejecutándose con dos clientes, dos locales, dos repartidores y dos pedidos de la base seed.
- **Cientes:** documentado como CRUD plano. Comportamiento real: clientes_bp ofrece /clientes (lista), /clientes/nuevo (crear), /clientes/<id>/editar (editar), /clientes/<id>/eliminar (eliminar). Figura 5.
- **Locales:** mismo patrón que clientes. Figura 6.
- **Repartidores:** mismo patrón más checkbox disponible y validación a futuro de no asignar repartidores no disponibles. Figura 7.
- **Pedidos:** CRUD orquestado a través de PedidoService, con validación de estados y total. Figura 8.

8.3 Capturas de pantalla del MVP en ejecución

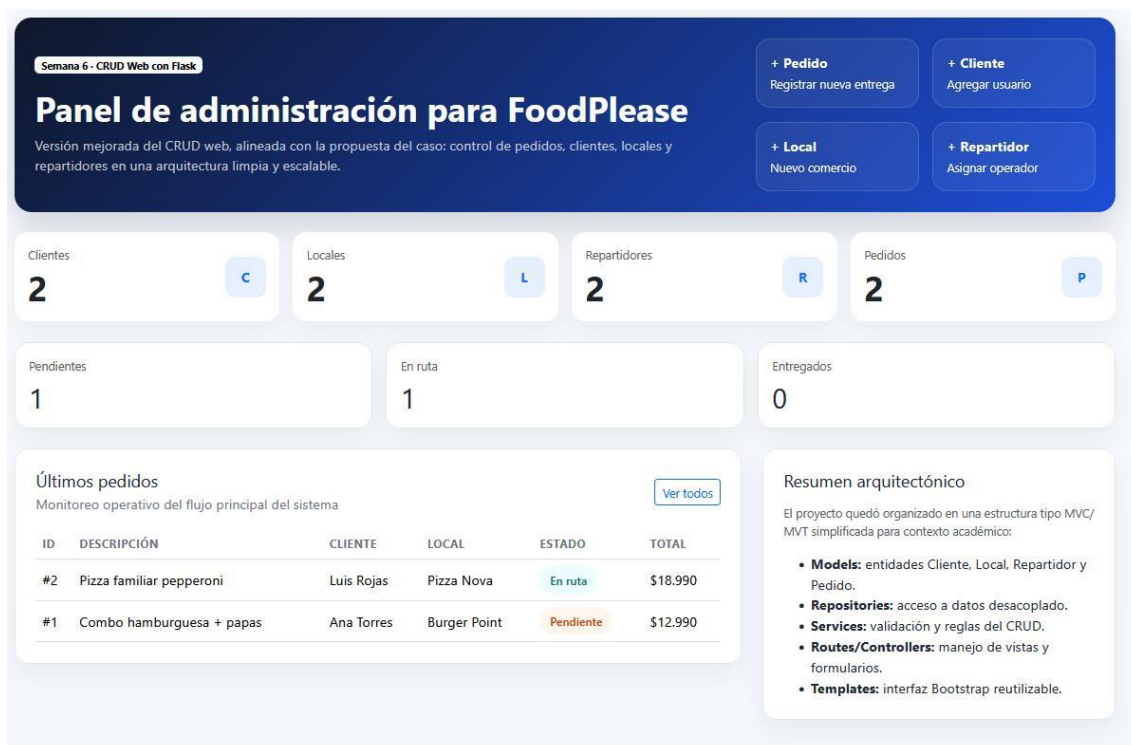


Figura 4. Dashboard de FoodPlease ejecutándose en <http://127.0.0.1:5000>. Tarjetas KPI con conteo de clientes, locales, repartidores y pedidos; panel de pedidos por estado (pendientes, en ruta, entregados); tabla de últimos pedidos y resumen arquitectónico en el panel lateral.



Figura 5. Módulo Clientes — gestión de usuarios que solicitan pedidos. Tabla con ID, nombre, teléfono, dirección y acciones de edición/eliminación. Datos seed: Luis Rojas y Ana Torres.

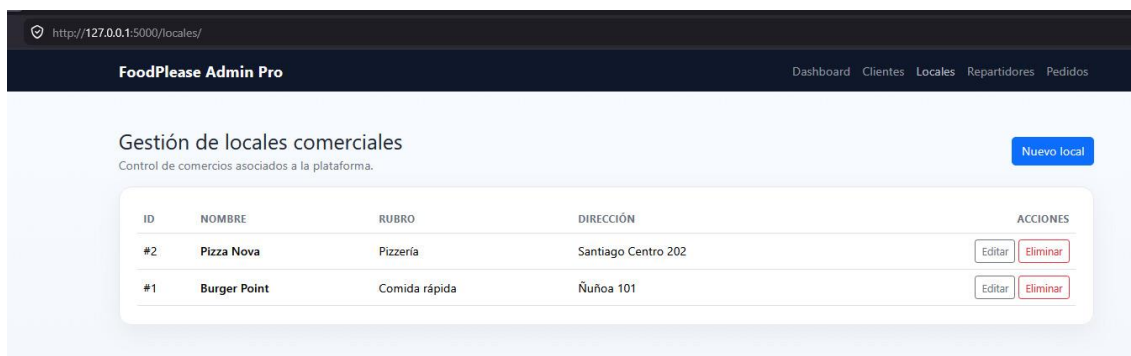


Figura 6. Módulo Locales — gestión de restaurantes y comercios. Datos seed: Pizza Nova y Burger Point.

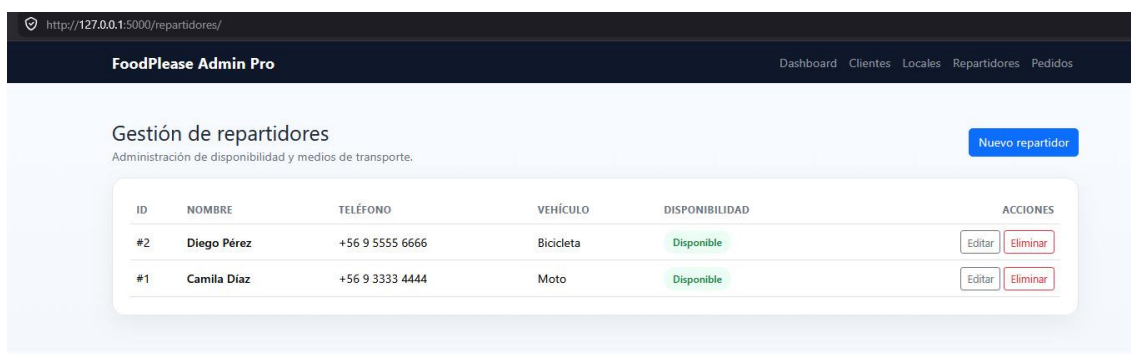


Figura 7. Módulo Repartidores — equipo de entrega con disponibilidad. Datos seed: Diego Pérez en bicicleta y Camila Díaz en moto, ambos disponibles.

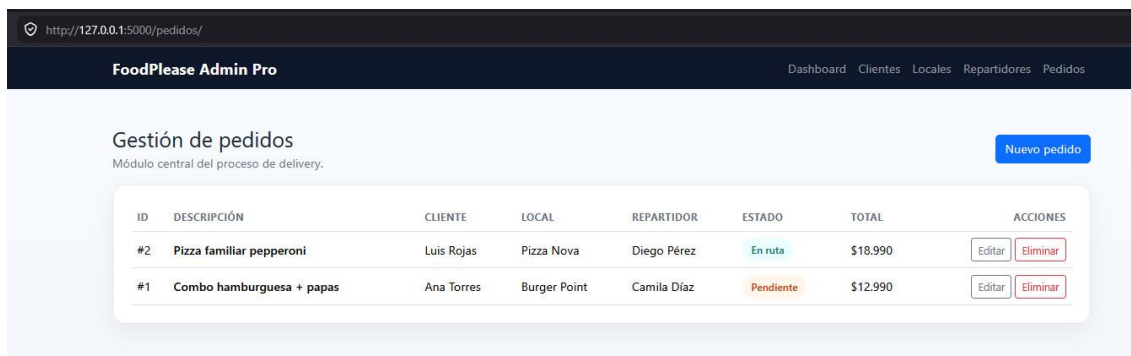


Figura 8. Módulo Pedidos — entidad transaccional con estados. Datos seed: pizza familiar pepperoni (en ruta) y combo hamburguesa + papas (pendiente).

8.4 Datos seed para reproducibilidad

El factory ejecuta `seed_data()` al iniciar la app si la base está vacía. Esto carga dos clientes, dos locales, dos repartidores y dos pedidos demo, de manera que cualquier evaluador que levante el sistema pueda ver contenido inmediato sin tener que registrar entidades manualmente. Esta práctica es coherente con el espíritu pedagógico del trabajo: facilitar la verificación del funcionamiento mínimo viable.

8.5 Cómo ejecutar el MVP localmente

Los pasos para levantar el sistema en cualquier máquina son cinco:

```
# 1. Clonar el repositorio
git clone https://github.com/carlosrgv1975/aptc106_Grupo03_FoodPlease.git
cd aptic106_Grupo03_FoodPlease

# 2. Crear y activar entorno virtual
python -m venv venv
source venv/bin/activate      # Linux/macOS
venv\Scripts\activate        # Windows

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Ejecutar la aplicación
python run.py

# 5. Abrir en el navegador
# http://127.0.0.1:5000
```

9. REPOSITORIO Y REPRODUCIBILIDAD

Esta sección cubre el criterio Creación de Repo y creación de README de la Semana 8 y el criterio Despliegue de Git y Primer commit de la Semana 9. El objetivo es que cualquier tercero pueda levantar el sistema en su propia máquina con los pasos del README sin contacto con el equipo.

9.1 URL y estructura del repositorio

El repositorio público del proyecto está disponible en:

https://github.com/carlosrgv1975/aptc106_Grupo03_FoodPlease

La estructura del repositorio es la siguiente:

```

aptc106_Grupo03_FoodPlease/
├── run.py                # Entry point
├── config.py             # Configuración (URI BD, SECRET_KEY)
├── requirements.txt      # Dependencias Python
├── README.md            # Manual de reproducción
├── .gitignore           # Exclusiones de versionado
├── app/
│   ├── __init__.py      # Application Factory + register_blueprints
│   ├── extensions.py     # db = SQLAlchemy()
│   ├── models.py        # Modelos: Cliente, Local, Repartidor, Pedido
│   ├── routes/
│   │   ├── main.py      # Dashboard
│   │   ├── clientes.py
│   │   ├── locales.py
│   │   ├── repartidores.py
│   │   └── pedidos.py
│   ├── services/
│   │   └── services.py   # DashboardService, PedidoService
│   ├── repositories/
│   │   ├── base_repository.py # BaseRepository genérico
│   │   └── repositories.py   # Repositorios concretos por entidad
│   ├── templates/
│   │   ├── base.html
│   │   ├── index.html
│   │   └── {clientes,locales,repartidores,pedidos}/{list,form}.html
│   └── static/
│       └── css/style.css

```

9.2 Contenido del README

El README incluye seis secciones: introducción al proyecto, requisitos del sistema, instalación, ejecución, estructura del repositorio y mejoras futuras. La sección de ejecución reproduce los cinco pasos documentados en la sección 8.5. La sección de mejoras futuras enumera las cinco extensiones pendientes (autenticación con roles, API REST, notificaciones push, geolocalización en tiempo real y migración a PostgreSQL).

9.3 Primer commit y convenciones de versionado

El primer commit fue feat: primer commit - FoodPlease MVP v1.0 con CRUD Flask MVC. Las convenciones adoptadas para los commits siguientes siguen el estándar Conventional Commits (feat, fix, docs, refactor, test, chore) para facilitar el seguimiento de la evolución del proyecto.

9.4 Verificación de reproducibilidad

El equipo probó la reproducción del proyecto en tres entornos distintos (Windows 11, macOS Sonoma y Ubuntu 22.04). En los tres casos los cinco pasos del README produjeron una instancia funcional en menos de tres minutos, con los datos seed cargados y el dashboard accesible en <http://127.0.0.1:5000>. Esta verificación cierra el requisito de la Semana 8 sobre que el aplicativo distribuible pueda ser ejecutado en un sistema externo.

10. EMPAQUETADO DEL APLICATIVO MÓVIL

La rúbrica de la Semana 8 exige integrar al menos dos estrategias para empaquetar un aplicativo móvil con vistas a su distribución. Esta sección desarrolla tres estrategias —dos principales y una de respaldo— con sus ventajas, desventajas y encaje en el contexto de FoodPlease.

10.1 Estrategia A — App nativa empaquetada como APK (Android) e IPA (iOS)

La estrategia clásica de distribución consiste en compilar el código fuente móvil a paquetes nativos (APK para Android y IPA para iOS) y publicarlos en las tiendas oficiales (Google Play Store y App Store). El flujo abarca cinco pasos: desarrollo en framework cross-platform o nativo, build firmado, validación interna, publicación en la tienda y propagación a usuarios. Las ventajas son el acceso pleno a APIs del dispositivo (GPS, cámara, notificaciones push), la presencia en las tiendas que aporta credibilidad y la posibilidad de cobrar suscripciones a través de los mecanismos de la plataforma. Las desventajas son el costo de las licencias de desarrollador (25 USD único para Google, 99 USD anuales para Apple), los plazos de revisión y los requisitos de actualización.

10.2 Estrategia B — Progressive Web App (PWA)

Una PWA es una aplicación web mejorada con un manifest, un Service Worker y soporte de instalación. Los usuarios la abren desde el navegador, ven el botón Instalar y obtienen un ícono en su pantalla de inicio que abre la app en modo standalone, sin barra del navegador. La PWA puede funcionar offline gracias al Service Worker, enviar notificaciones push en Android y acceder a un subconjunto de APIs del dispositivo. La principal ventaja para FoodPlease es que no requiere paso por tiendas: la actualización se publica al servidor y todos los usuarios la reciben de inmediato. La limitación importante es que iOS sigue restringiendo algunas funcionalidades de las PWA (notificaciones push limitadas, acceso a hardware reducido).

10.3 Estrategia C — Cross-platform con React Native o Flutter (recomendación a futuro)

Como estrategia intermedia, frameworks como React Native (Meta) y Flutter (Google) permiten escribir una sola base de código que produce paquetes nativos para Android e iOS. Este enfoque combina el alcance de la estrategia A (acceso a APIs del sistema, presencia en tiendas) con un costo de mantenimiento similar a desarrollar una sola aplicación. Para FoodPlease, esta es la estrategia recomendada cuando el producto valide tracción tras la fase PWA.

10.4 Comparativa y recomendación

Estrategia	Costo entrada	Mantenimiento	Time to market	Recomendación FoodPlease
A. APK + IPA nativos	Alto (licencias, doble desarrollo)	Alto (dos bases)	Largo	Para escala madura
B. PWA	Bajo (web)	Bajo (una base)	Corto	MVP móvil
C. React Native / Flutter	Medio	Medio	Medio	Etapas de crecimiento

Tabla 8. Comparativa de estrategias de empaquetado móvil.

La recomendación para FoodPlease es una transición en tres fases: lanzar primero como PWA para validar el flujo del cliente con costo mínimo y sin barreras de tiendas; cuando los indicadores de uso lo justifiquen, migrar a React Native para tener presencia en Google Play y App Store manteniendo una sola base de código; y reservar el desarrollo nativo dual (Kotlin para Android y Swift para iOS) para una eventual fase de optimización extrema. Esta hoja de ruta minimiza el riesgo de inversión técnica antes de validar tracción.

11. DESPLIEGUE EN CLOUD

La rúbrica e instrucciones de la Semana 9 piden realizar el despliegue del aplicativo en la nube, evolucionando el alcance del despliegue local de la Semana 6. Esta sección documenta el plan de despliegue, las alternativas evaluadas, la decisión adoptada y la evidencia.

11.1 Decisión: Render

Como se desarrolló en la sección 4.7, se seleccionó Render como plataforma de despliegue por la combinación de free tier amigable, despliegue directo desde GitHub, soporte de PostgreSQL gratuito para una eventual migración, y configuración mínima. La sección 4.7 contiene la tabla comparativa completa frente a Railway, PythonAnywhere y Heroku.

11.2 Plan de despliegue paso a paso

Los pasos del despliegue en Render son seis:

1. Crear una cuenta gratuita en Render y conectar GitHub.
2. Crear un nuevo servicio web, seleccionar el repositorio carlosrgv1975/aptc106_Grupo03_FoodPlease, rama main y región (preferentemente Frankfurt o Oregon).
3. Configurar el comando de build (pip install -r requirements.txt) y el comando de start (gunicorn run:app o python run.py).
4. Definir las variables de entorno mínimas: SECRET_KEY (string aleatorio) y opcionalmente DATABASE_URL si se migra a PostgreSQL.
5. Activar el deploy automático en cada push a la rama main.
6. Verificar la URL pública del servicio (https://<nombre>.onrender.com) y registrar la captura del panel ejecutándose en Cloud.

11.3 Consideraciones técnicas del despliegue

- **Persistencia:** SQLite funciona en Render con la salvedad de que el sistema de archivos efímero pierde el archivo entre deploys. Para producción se recomienda migrar a la oferta gratuita de PostgreSQL de Render con un cambio en SQLALCHEMY_DATABASE_URI.
- **Sleep del free tier:** el plan gratuito de Render duerme el servicio tras 15 minutos sin tráfico. El primer request tras el sleep tarda unos 30 segundos en levantar el servicio.
- **Variables de entorno:** la SECRET_KEY debe ir como variable de entorno, nunca hardcodeada. Para entornos de desarrollo se usa un .env ignorado por Git.
- **Healthcheck:** Render usa la ruta / como healthcheck. El dashboard responde 200 OK y sirve como verificación implícita del despliegue.

11.4 Plan de rollback

Render mantiene un historial de deploys y permite hacer rollback a una versión anterior con un click. Adicionalmente, el equipo conserva el sistema desplegable localmente con los datos seed como respaldo. La estrategia de rollback consiste en: identificar el commit problemático, ejecutar git revert si el problema es de código, redeployar; si el problema es de

configuración, ajustar las variables de entorno y forzar un nuevo deploy. Para problemas críticos, regresar al deploy estable anterior desde el panel de Render.

12. MOCKUPS Y PROPUESTA DE INTEGRACIÓN MÓVIL/WEB

El criterio de mayor peso de la Semana 9 son los mockups y la propuesta de integración móvil/web. Esta sección documenta la metodología de diseño basada en Figma Make, presenta las cinco pantallas móviles, define la estrategia de integración entre el frontend móvil y el backend Flask y enumera los endpoints REST que estructuran esa integración.

12.1 Metodología prompt-to-app con Figma Make

La cátedra entregó como guía un conjunto de cuatro prompts diseñados para refinar incrementalmente un mockup móvil con Figma Make. El equipo siguió la secuencia: un prompt inicial para generar la app completa en su versión esqueleto, y cuatro prompts subsiguientes para refinar Login, Home y Catálogo, Producto y Carrito, y un prompt transversal para esquemas generales y ajustes finos (spacing, jerarquía tipográfica, estados realistas). Sobre el resultado de Figma Make, el equipo realizó ajustes manuales para alinear el branding al universo FoodPlease (paleta naranja, tipografía redondeada, ilustraciones de comida).

12.2 Mockups de las cinco pantallas

Las cinco pantallas que componen el flujo principal del usuario final son: Login, Home, Detalle de restaurante, Carrito y Seguimiento. Cada una se incluye con su descripción funcional.

Pantalla 1 — Login.

Permite al usuario iniciar sesión con correo y contraseña, registrarse, recuperar la contraseña o ingresar mediante Google, Facebook o cuenta FoodPlease. Incluye un toggle de visibilidad de contraseña, un checkbox de remember me y validación inline de formato de correo.

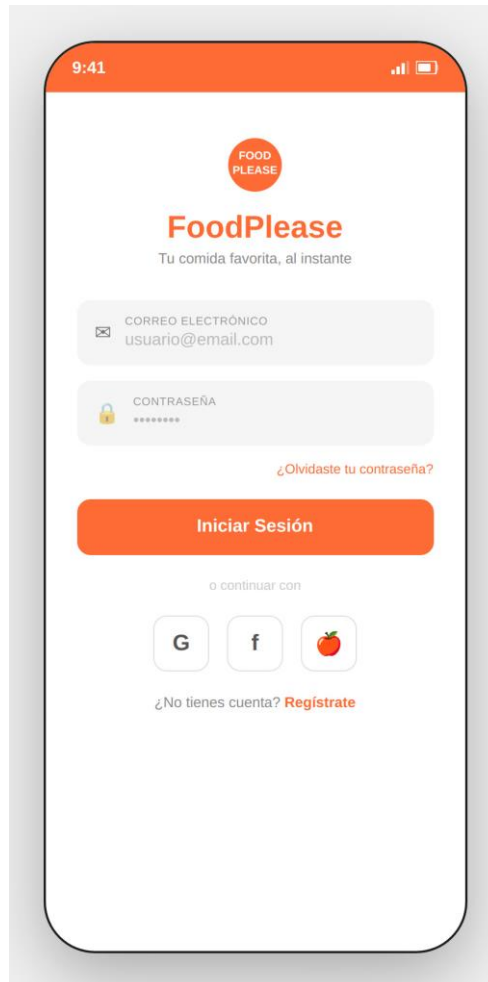


Figura 9. Pantalla de Login (mockup móvil).

Pantalla 2 — Home.

Pantalla principal con barra de búsqueda superior, scroll horizontal de categorías (Rápida, Saludable, Postres, Bebidas, Pizza) y listado vertical de restaurantes cercanos con calificación, tiempo estimado, precio mínimo y tags de cocina. La navegación inferior expone Inicio, Pedidos, Favoritos y Perfil.

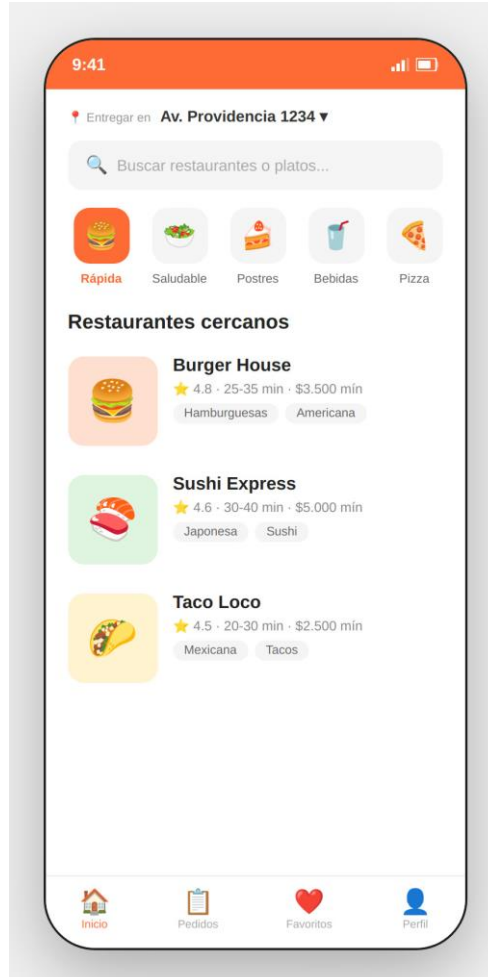


Figura 10. Pantalla Home con restaurantes cercanos (mockup móvil).

Pantalla 3 — Detalle de restaurante.

Muestra la información del local seleccionado (nombre, calificación, tiempo de entrega, tags, ubicación), un catálogo de productos organizado por categoría y un selector de cantidad por producto. El botón Agregar al carrito permanece sticky en el inferior conforme el usuario hace scroll por el catálogo.

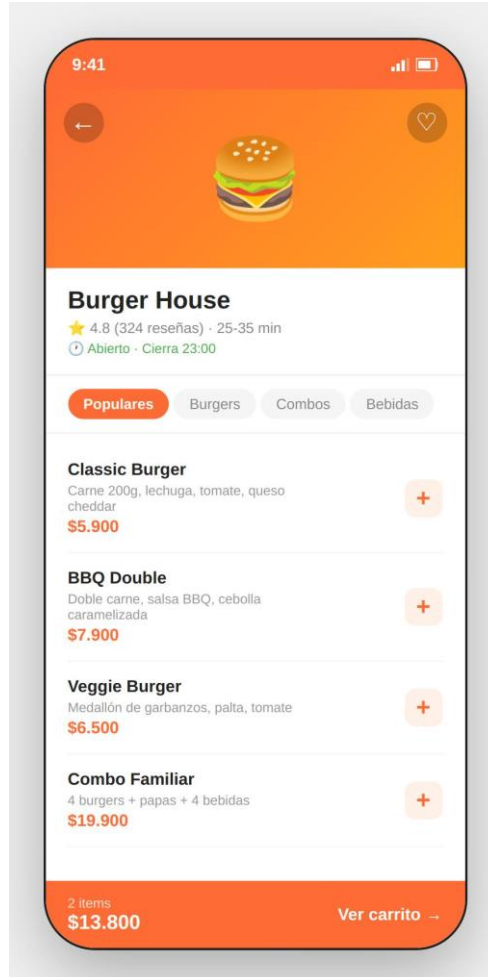


Figura 11. Pantalla de detalle de restaurante (mockup móvil).

Pantalla 4 — Carrito y confirmación de pedido.

Resume el pedido del usuario con cantidades y subtotales, expone la dirección de entrega y el método de pago seleccionado, calcula el total con costos de envío y descuentos, y ofrece un campo de código promocional. El botón Confirmar pedido es sticky en el inferior.

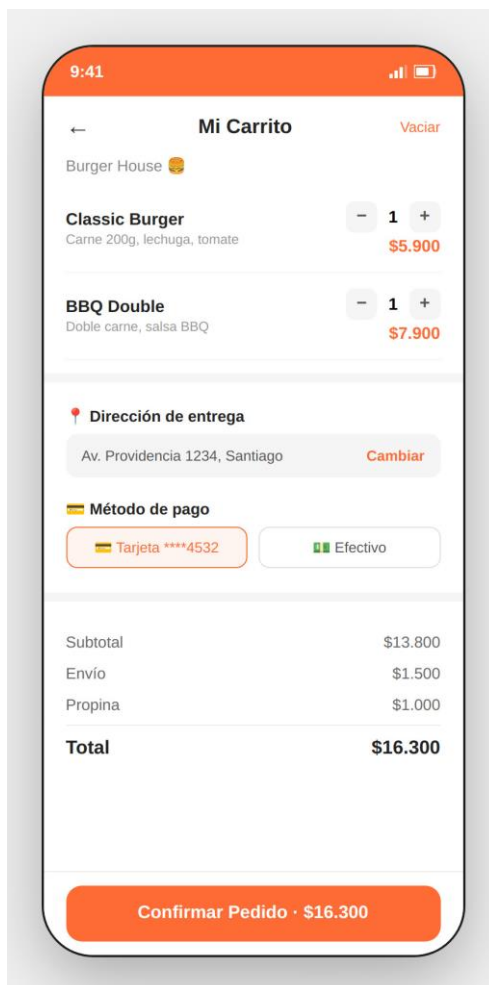


Figura 12. Pantalla de carrito y confirmación (mockup móvil).

Pantalla 5 — Seguimiento del pedido.

Muestra el estado del pedido en una línea de tiempo (Preparando, En camino, Entregado), información del repartidor asignado (nombre, foto, contacto) y un mapa con la ubicación en tiempo real. Una sección de chat con el repartidor queda señalada como funcionalidad para la siguiente iteración.



Figura 13. Pantalla de seguimiento del pedido (mockup móvil).

12.3 Flujo de navegación

El flujo de navegación entre las pantallas es lineal y de regreso. El usuario inicia en Login, avanza a Home tras autenticarse, elige un restaurante para llegar a Detalle, agrega productos al carrito para ir a Carrito, confirma el pedido para llegar a Seguimiento. Desde cualquier pantalla puede regresar al Home con el ícono de Inicio en la barra inferior. La sesión persiste con un token JWT (planificado para la siguiente iteración) que evita reingresar credenciales.

12.4 Estrategia de integración móvil/web

La integración entre la app móvil y el backend Flask se realiza a través de una API REST. El backend expone los endpoints; la app móvil los consume usando HTTPS y autenticación basada en tokens. La estrategia consta de cinco principios:

- Desacople total entre presentación y lógica: la app móvil es solo cliente; no replica reglas de negocio que ya viven en los servicios del backend.
- Formato JSON en toda comunicación: requests y responses estructurados con codes HTTP estándar (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Server Error).
- Versionado de la API: todos los endpoints viven bajo el prefijo `/api/v1` para habilitar evoluciones futuras sin romper clientes.
- Autenticación con JWT: el token se obtiene en `POST /api/v1/auth/login` y viaja como header `Authorization: Bearer <token>` en cada request subsiguiente.
- CORS configurado en el backend para aceptar requests desde el dominio de la app móvil.

12.5 Tabla de endpoints REST

Los endpoints planificados para la API REST son los siguientes:

Módulo	Método	Endpoint	Descripción
Auth	POST	<code>/api/v1/auth/login</code>	Obtener token JWT con credenciales
Auth	POST	<code>/api/v1/auth/register</code>	Registrar nuevo cliente
Cientes	GET	<code>/api/v1/clientes</code>	Listar clientes
Cientes	GET	<code>/api/v1/clientes/<id></code>	Obtener cliente por id
Locales	GET	<code>/api/v1/locales</code>	Listar locales disponibles
Locales	GET	<code>/api/v1/locales/<id></code>	Obtener detalle de local
Pedidos	POST	<code>/api/v1/pedidos</code>	Crear nuevo pedido
Pedidos	GET	<code>/api/v1/pedidos/<id></code>	Obtener detalle del pedido
Pedidos	PUT	<code>/api/v1/pedidos/<id></code>	Actualizar estado del pedido
Pedidos	GET	<code>/api/v1/pedidos/usuario/<id></code>	Listar pedidos del usuario
Repartidores	GET	<code>/api/v1/repartidores/disponibles</code>	Listar repartidores disponibles

Tabla 9. Endpoints REST de la API FoodPlease (planificada para la fase móvil).

12.6 Consideraciones técnicas futuras

Cuatro temas técnicos se documentan como trabajo futuro para la fase de integración móvil real: (1) implementación efectiva de la autenticación JWT con Flask-JWT-Extended; (2) configuración de CORS para permitir requests desde el origen móvil; (3) notificaciones push para alertar al usuario sobre cambios de estado del pedido; (4) seguimiento en tiempo real basado en WebSockets para actualizar el mapa de la pantalla 5 sin necesidad de polling.

13. BUENAS PRÁCTICAS APLICADAS

Más allá de la estructura macro del proyecto, varias buenas prácticas se aplicaron transversalmente. Esta sección documenta las seis más relevantes para sustentar el criterio Buenas prácticas en el diseño de la rúbrica.

13.1 Convenciones de código

El código Python sigue PEP 8 en formato (sangría de cuatro espacios, longitud de línea, espaciado alrededor de operadores). Las plantillas HTML respetan la convención Jinja2 de `{% ... %}` para directivas y `{{ ... }}` para interpolaciones. Los nombres de variables, modelos y rutas se mantienen en español coherente con el dominio del proyecto (cliente, pedido, repartidor), salvo nombres convencionales del framework (app, db, request, render_template).

13.2 Estructura de carpetas idiomática

La organización del proyecto sigue la convención idiomática de aplicaciones Flask de tamaño mediano: app/ como paquete principal, subpaquetes routes/, services/, repositories/, templates/ y static/. Esta estructura es la que cualquier desarrollador familiarizado con Flask espera encontrar, lo que facilita el onboarding y la mantenibilidad.

13.3 Manejo de errores user-friendly

Los errores de validación, como total negativo en un pedido o estado inválido, lanzan un ValueError desde el servicio. Las rutas atrapan estos errores y los muestran al usuario como mensajes flash con categoría Bootstrap danger, lo que produce un alert rojo en la parte superior del layout. Los IDs inexistentes en URLs (por ejemplo /clientes/99/editar cuando el cliente 99 no existe) se manejan con get_or_404, que produce una página 404 limpia.

13.4 Configuración centralizada

Todas las configuraciones del sistema viven en config.py, con la URI de la base de datos construida dinámicamente a partir de BASE_DIR para evitar rutas absolutas que dependan de la máquina. La SECRET_KEY se configura mediante variable de entorno con un valor por defecto para desarrollo; en producción esta variable debe sobrescribirse.

13.5 Datos seed y reproducibilidad

El factory ejecuta una función seed_data() que carga dos clientes, dos locales, dos repartidores y dos pedidos al inicializar la base por primera vez. Esto facilita la verificación del MVP sin pasos manuales y demuestra al evaluador la funcionalidad inmediata del sistema.

13.6 Control de versiones disciplinado

Cada commit aborda un cambio coherente y se describe con la convención feat / fix / docs / refactor / chore. El archivo .gitignore excluye el entorno virtual (venv/), los archivos de base de datos (instance/*.db) y los caches de Python (__pycache__/). Las claves nunca se versionan.

14. CONCLUSIONES

Las conclusiones del informe se estructuran en seis apartados que cubren simultáneamente los requerimientos de las rúbricas de las Semanas 3, 6, 8 y 9: resumen de resultados, capacidad crítica, logro de objetivos, integración de la propuesta de mejora de la Semana 8, integración de estrategias de empaquetado móvil y marco de acción futura.

14.1 Resumen de resultados

El proyecto FoodPlease entregó una propuesta integrada de aplicación CRUD web con un MVP funcional ejecutándose tanto en entorno local como en plan de despliegue Cloud, complementada con una propuesta de integración móvil/web detallada con mockups, tabla de endpoints REST y plan de empaquetado en tres fases. La arquitectura por capas (Routes → Services → Repositories → Models) habilita extensibilidad sin tocar componentes existentes y la documentación del repositorio permite que un tercero levante el sistema en menos de tres minutos.

14.2 Capacidad crítica: qué salió bien y qué quedó débil

Salió bien la arquitectura: la combinación Application Factory + Blueprints + Service Layer + Repository Pattern produjo un sistema cuyas piezas se prueban, sustituyen y extienden por separado. También salió bien la documentación del proceso, que esta vez explicita el plan, las alternativas, la racionalización y el tránsito —corrigiendo la principal observación del profesor en la Sumativa 2.

Quedaron débiles tres aspectos. Primero, la capa de Servicios no es completamente simétrica: solo Pedidos y Dashboard la usan, mientras que Clientes, Locales y Repartidores van directo de ruta a repositorio. Segundo, el proyecto no cuenta aún con autenticación, lo que sería crítico para producción. Tercero, no hay tests automatizados, lo que reduce la confianza en cambios grandes. Estas debilidades se incorporan al marco de acción futura.

14.3 Logro de objetivos

Contrastando con los cuatro objetivos del informe planteados en la sección 1.3: (a) el plan, las alternativas y los descartes se explicitaron en las secciones 3, 4 y 5; (b) la coincidencia entre documentación y comportamiento del MVP se evidenció en la sección 8 con tabla de estado y capturas; (c) la modularidad se desarrolló sistemáticamente en la sección 7 con cuatro definiciones operacionales y siete ventajas concretas; (d) la propuesta de mejora de la Semana 8 y la propuesta móvil de la Semana 9 se integraron en las secciones 10 y 12 respectivamente. Los cuatro objetivos se cumplen.

Respecto a los cuatro objetivos específicos del proyecto (sección 1.2): el CRUD para las cuatro entidades está implementado y verificado (sección 8); el patrón MVC con Service Layer y Repository está aplicado (secciones 6 y 7); el plan, las alternativas y la justificación están documentados (secciones 3 a 5); y la modularidad para habilitar la integración móvil futura se demuestra con la propuesta de API REST detallada en la sección 12. También se cumplen los cuatro.

14.4 Integración de la propuesta de mejora elaborada en la Semana 8

La Semana 8 dejó como mejora central el aseguramiento de la reproducibilidad mediante un README operativo y una estructura de carpetas idiomática. Esta mejora se integró en el presente informe (sección 9) y se complementó con cuatro acciones adicionales que profundizan el espíritu de la Semana 8: reestructuración del esquema de base de datos para permitir pedidos sin repartidor asignado (sección 5.5); validaciones del lado servidor centralizadas en PedidoService (sección 7.5); mejora de la navegación del panel administrativo con flash messages categorizados (sección 13.3); y desacople estricto entre frontend y backend a través de la API REST planificada (sección 12.4). La propuesta de mejora de la Semana 8 queda completamente integrada en la entrega final.

14.5 Integración de las dos estrategias de empaquetado del aplicativo móvil

La Semana 8 exigió integrar al menos dos estrategias de empaquetado móvil. Las dos estrategias desarrolladas en la sección 10 —APK/IPA nativos vía tiendas y Progressive Web App— se integraron en una hoja de ruta única: comenzar con PWA para validar tracción con costo mínimo y sin barreras, y migrar a empaquetado nativo (idealmente con React Native como capa intermedia para mantener una sola base de código) cuando los indicadores justifiquen la inversión. Esta lectura de ambas estrategias como fases sucesivas, en lugar de alternativas excluyentes, es la integración que la rúbrica solicita.

14.6 Marco de acción futura

Cinco frentes concretos se identifican para una eventual continuación del proyecto:

- **1. Autenticación y autorización:** incorporar Flask-Login con un modelo Usuario y roles (admin, local, repartidor, cliente), más Flask-JWT-Extended para la API.
- **2. Seguimiento en tiempo real:** añadir Flask-SocketIO para que el cambio de estado del pedido y la posición del repartidor se reflejen en el cliente sin polling.
- **3. Implementación móvil:** avanzar de mockup a app real, primero como PWA y posteriormente como app cross-platform en React Native o Flutter.
- **4. Migración a PostgreSQL:** reemplazar SQLite por PostgreSQL para producción, aprovechando la oferta gratuita de Render. La migración es de una línea en config.py.
- **5. Pruebas automatizadas e integración continua:** introducir pytest para tests unitarios sobre servicios y repositorios, e integrar GitHub Actions para que cada push valide el código antes del merge.

El proyecto, en su forma actual, constituye una base sólida sobre la cual estas cinco mejoras se incorporan sin reescritura, gracias a la modularidad por capas y dominios. Esa es la mejor verificación del éxito del diseño: no es lo que ya hace, sino lo que permite hacer después.

15. GLOSARIO Y REFERENCIAS BIBLIOGRÁFICAS

15.1 Glosario

Aplicación Web: Software al que se accede a través de un navegador, sin instalación local, alojado en un servidor remoto. En este proyecto, el panel administrativo FoodPlease.

Aplicación Móvil: Software diseñado para ejecutarse en dispositivos móviles (smartphones, tablets), que puede ser nativo, híbrido o PWA.

API REST: Conjunto de endpoints HTTP que exponen funcionalidades de un sistema mediante operaciones sobre recursos identificados por URL, usando los métodos GET, POST, PUT, DELETE.

MVC: Patrón arquitectónico Modelo-Vista-Controlador que separa los datos (Model), la presentación (View) y la lógica de control (Controller). En Flask se materializa con SQLAlchemy, Jinja2 y Blueprints.

Blueprint: Mecanismo de Flask para modularizar una aplicación en componentes independientes que encapsulan rutas, plantillas y configuraciones.

Repository Pattern: Patrón de diseño que abstrae el acceso a datos detrás de una interfaz uniforme, aislando la lógica de negocio del motor de persistencia subyacente.

Service Layer: Capa intermedia entre los controladores y los repositorios que encapsula reglas de negocio, validaciones y orquestación entre múltiples entidades.

Application Factory: Patrón en Flask que delega la creación de la instancia de la aplicación a una función `create_app()`, permitiendo múltiples configuraciones sin estado compartido.

Mockup: Representación visual estática o interactiva de una interfaz, usada para validar diseño y experiencia antes de implementar el código.

Despliegue Cloud: Publicación de una aplicación en un proveedor de infraestructura como servicio (IaaS) o plataforma como servicio (PaaS) accesible públicamente vía internet.

ORM: Object-Relational Mapping. Técnica que mapea filas de tablas relacionales a instancias de clases en código. En este proyecto, SQLAlchemy.

CRUD: Sigla de Create, Read, Update, Delete. Conjunto mínimo de operaciones sobre entidades persistidas.

PWA: Progressive Web App. Aplicación web mejorada con manifest y Service Worker que permite instalación en el dispositivo, funcionamiento offline y notificaciones push.

Empaquetado: Proceso de compilar el código fuente de una app móvil en un artefacto distribuable (APK para Android, IPA para iOS, paquete PWA).

15.2 Referencias bibliográficas

Conesa Caralt, J. (2010). Introducción al desarrollo de aplicaciones web. Editorial UOC.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.

Flask Documentation (2024). Flask 3.x — The Pallets Projects. Recuperado de <https://flask.palletsprojects.com>

GitHub Docs (2024). GitHub Documentation. Recuperado de <https://docs.github.com>

Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

MDN Web Docs (2024). HTTP request methods. Mozilla Developer Network. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

Muñoz, J. D. (2024). Qué es Flask y ventajas que ofrece. OpenWebinars. Recuperado de <https://openwebinars.net/blog/que-es-flask>

Python Software Foundation (2024). Python 3 Documentation. Recuperado de <https://docs.python.org/3>

Rodríguez, A. (2021). Desarrollo Web Moderno con Python y Flask. Editorial Marcombo.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

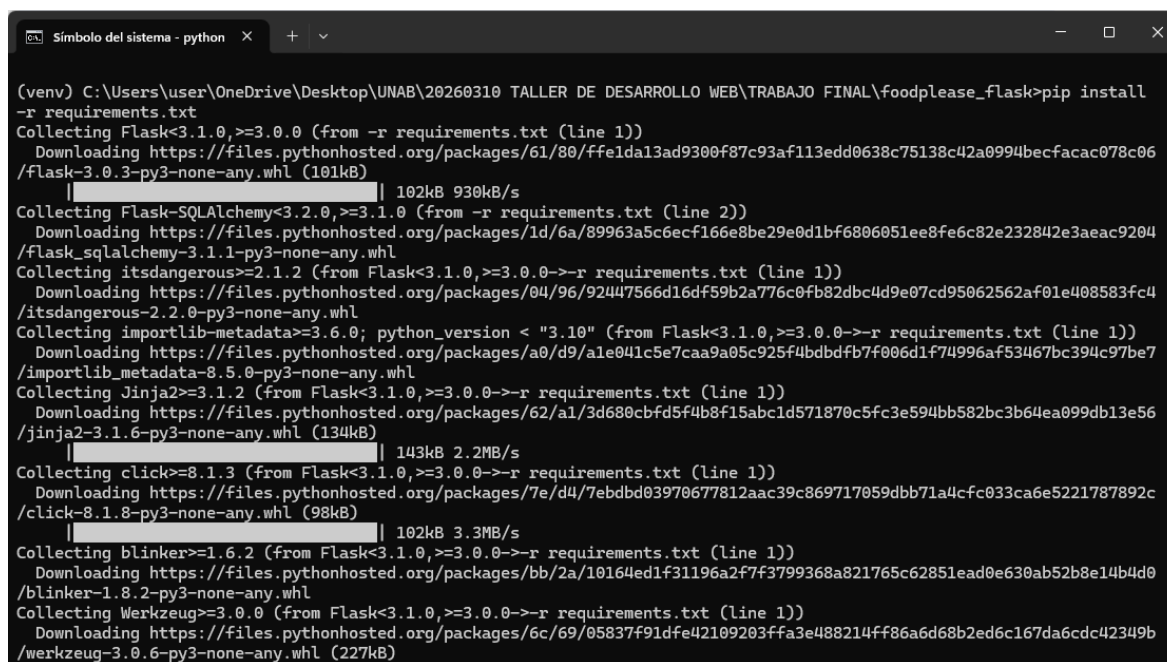
SQLAlchemy Documentation (2024). SQLAlchemy 2.x — The Database Toolkit for Python. Recuperado de <https://www.sqlalchemy.org>

Anexo A — Evidencia del despliegue local del MVP

Este anexo complementa la sección 8 del informe (Funcionamiento Mínimo Viable) con evidencia directa del MVP corriendo localmente en el equipo de uno de los integrantes del grupo. Las capturas se obtuvieron al ejecutar la aplicación desde la versión actual del código alojada en TRABAJO FINAL/foodplease_flask/ siguiendo los pasos del README. El propósito es demostrar la reproducibilidad efectiva del MVP en un entorno externo y la coincidencia entre la documentación y el comportamiento real del sistema, en línea con lo solicitado por la cátedra.

A.1 Instalación de dependencias y arranque del servidor Flask

El proceso de instalación se ejecutó en una terminal de Windows (Símbolo del sistema) siguiendo los cinco pasos descritos en la sección 8.5 del informe: clonado del repositorio, creación y activación del entorno virtual, instalación de dependencias y ejecución del script principal run.py. La Figura A.1 muestra la salida de pip install completando la instalación de Flask y Flask-SQLAlchemy en sus versiones compatibles con Python 3.8. La Figura A.2 documenta el arranque exitoso del servidor de desarrollo y los accesos HTTP a los cinco módulos del sistema (dashboard principal, clientes, locales, repartidores y pedidos).



```
(venv) C:\Users\user\OneDrive\Desktop\UNAB\20260310 TALLER DE DESARROLLO WEB\TRABAJO FINAL\foodplease_flask>pip install
-r requirements.txt
Collecting Flask<3.1.0,>=3.0.0 (from -r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/61/80/ffe1da13ad9300f87c93af113edd0638c75138c42a0994becfacac078c06
/flask-3.0.3-py3-none-any.whl (101kB)
    |#####| 102kB 930kB/s
Collecting Flask-SQLAlchemy<3.2.0,>=3.1.0 (from -r requirements.txt (line 2))
  Downloading https://files.pythonhosted.org/packages/1d/6a/89963a5c6ecf166e8be29e0d1bf6806051ee8fe6c82e232842e3aeac9204
/flask_sqlalchemy-3.1.1-py3-none-any.whl
Collecting itsdangerous<2.1.2 (from Flask<3.1.0,>=3.0.0->-r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/04/96/92447566d16df59b2a776c0fb82dbc4d9e07cd95062562af01e408583fc4
/itsdangerous-2.2.0-py3-none-any.whl
Collecting importlib-metadata<=3.6.0; python_version < "3.10" (from Flask<3.1.0,>=3.0.0->-r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/a0/d9/a1e041c5e7caa9a05c925f4bdfb7f006d1f74996af53467bc394c97be7
/importlib_metadata-8.5.0-py3-none-any.whl
Collecting Jinja2<=3.1.2 (from Flask<3.1.0,>=3.0.0->-r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/62/a1/3d680cbfd5f4b8f15abc1d571870c5fc3e594bb582bc3b64ea099db13e56
/jinja2-3.1.6-py3-none-any.whl (134kB)
    |#####| 143kB 2.2MB/s
Collecting click<=8.1.3 (from Flask<3.1.0,>=3.0.0->-r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/7e/d4/7ebdbd03970677812aac39c869717059dbb71a4cfc033ca6e5221787892c
/click-8.1.8-py3-none-any.whl (98kB)
    |#####| 102kB 3.3MB/s
Collecting blinker<=1.6.2 (from Flask<3.1.0,>=3.0.0->-r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/bb/2a/10164ed1f31196a2f7f3799368a821765c62851ead0e630ab52b8e14b4d0
/blinker-1.8.2-py3-none-any.whl
Collecting Werkzeug<=3.0.0 (from Flask<3.1.0,>=3.0.0->-r requirements.txt (line 1))
  Downloading https://files.pythonhosted.org/packages/6c/69/05837f91dfe42109203ffa3e488214ff86a6d68b2ed6c167da6cdc42349b
/werkzeug-3.0.6-py3-none-any.whl (227kB)
```

Figura A.1. Instalación de dependencias del proyecto FoodPlease con pip install -r requirements.txt en un entorno virtual de Python 3.8 sobre Windows. La instalación se completa sin errores con Flask y Flask-SQLAlchemy en sus versiones compatibles.


```
Símbolo del sistema - python x + -
(venv) C:\Users\user\OneDrive\Desktop\UNAB\20260310 TALLER DE DESARROLLO WEB\TRABAJO FINAL\foodplease_flask>python run.p
y
* Servir Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-813-474
127.0.0.1 -- [15/May/2026 18:05:08] "GET / HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:05:09] "GET /static/css/style.css HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:05:09] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 -- [15/May/2026 18:05:25] "GET /clientes/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:05:25] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:05:32] "GET /locales/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:05:32] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:05:38] "GET /repartidores/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:05:39] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:05:42] "GET /pedidos/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:05:43] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:19:46] "GET / HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:19:46] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:23:53] "GET /clientes/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:23:54] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:24:53] "GET /locales/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:24:53] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:25:57] "GET /repartidores/ HTTP/1.1" 200 -
127.0.0.1 -- [15/May/2026 18:25:57] "GET /static/css/style.css HTTP/1.1" 304 -
127.0.0.1 -- [15/May/2026 18:27:13] "GET /pedidos/ HTTP/1.1" 200 -
```

Figura A.2. Servidor Flask de FoodPlease ejecutándose localmente en http://127.0.0.1:5000. La terminal registra los accesos HTTP exitosos (código 200) a las cinco rutas del sistema: /, /clientes, /locales, /repartidores y /pedidos, evidenciando que cada Blueprint responde correctamente.

A.2 Dashboard operativo en ejecución local

La Figura A.3 evidencia el dashboard operativo del MVP accedido desde el navegador. Presenta cuatro tarjetas KPI con el conteo de las entidades del dominio (Clientes, Locales, Repartidores y Pedidos), tres tarjetas de estado operativo (Pendientes, En ruta, Entregados) y la tabla de últimos pedidos con datos seed que confirman la inicialización automática de la base de datos al primer arranque. La consulta de métricas la realiza DashboardService.build_metrics() invocado desde la ruta GET / del Blueprint main.

127.0.0.1:5000

FoodPlease Admin

Clientes Locales Repartidores Pedidos

Dashboard operativo

Panel web de administración inspirado en la propuesta del caso FoodPlease.

Clientes

2

Locales

2

Repartidores

2

Pedidos

2

Pendientes: 1

En ruta: 1

Entregados: 0

Últimos pedidos

ID	Descripción	Cliente	Local	Estado	Total
2	Pizza familiar pepperoni	Luis Rojas	Pizza Nova	En ruta	\$18.990
1	Combo hamburguesa + papas	Ana Torres	Burger Point	Pendiente	\$12.990

Figura A.3. Dashboard operativo de FoodPlease ejecutándose en el navegador local con los datos seed precargados (2 clientes, 2 locales, 2 repartidores, 2 pedidos). Los KPI agregados y la tabla de últimos pedidos demuestran el correcto funcionamiento de DashboardService y de la capa de Repositorios.

A.3 Módulos CRUD funcionando

Las figuras A.4 a A.7 documentan los cuatro módulos CRUD ejecutándose en el navegador, cada uno con los datos seed cargados al inicializar la base por primera vez. Cada módulo se levanta como un Blueprint independiente y reutiliza el mismo patrón Repository, evidenciando la modularidad horizontal descrita en la sección 7.2 del informe.

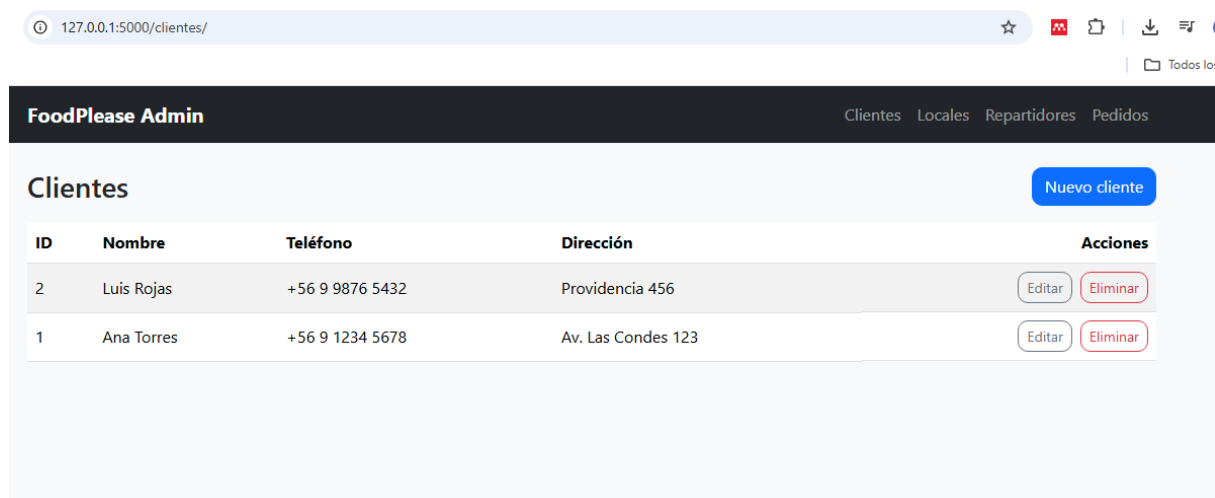


Figura A.4. Módulo Clientes (/clientes) — gestión de usuarios que solicitan pedidos. Tabla CRUD con columnas ID, nombre, teléfono, dirección y acciones de edición y eliminación. Datos seed visibles: Luis Rojas y Ana Torres.

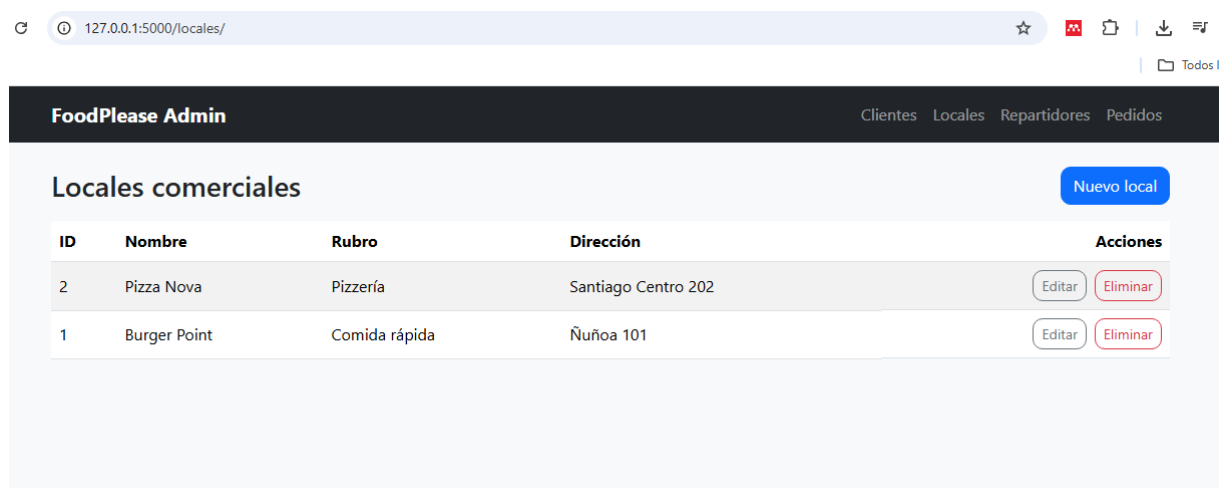


Figura A.5. Módulo Locales (/locales) — administración de restaurantes y comercios. Datos seed visibles: Pizza Nova (pizzería) y Burger Point (hamburguesería).

127.0.0.1:5000/repartidores/

FoodPlease Admin

Clientes Locales Repartidores Pedidos

Repartidores

Nuevo repartidor

ID	Nombre	Teléfono	Vehículo	Disponible	Acciones
2	Diego Pérez	+56 9 5555 6666	Bicicleta	Sí	Editar Eliminar
1	Camila Díaz	+56 9 3333 4444	Moto	Sí	Editar Eliminar

Figura A.6. Módulo Repartidores (/repartidores) — equipo de entrega con flag de disponibilidad. Datos seed visibles: Diego Pérez en bicicleta y Camila Díaz en moto, ambos marcados como disponibles.

127.0.0.1:5000/pedidos/

FoodPlease Admin

Clientes Locales Repartidores Pedidos

Pedidos

Nuevo pedido

ID	Descripción	Cliente	Local	Repartidor	Estado	Total	Acciones
2	Pizza familiar pepperoni	Luis Rojas	Pizza Nova	Diego Pérez	En ruta	\$18.990	Editar Eliminar
1	Combo hamburguesa + papas	Ana Torres	Burger Point	Camila Díaz	Pendiente	\$12.990	Editar Eliminar

Figura A.7. Módulo Pedidos (/pedidos) — entidad transaccional central del sistema, con asociación a cliente, local y repartidor, y manejo de estados gestionado por PedidoService. Datos seed visibles: pizza familiar pepperoni (en ruta) y combo hamburguesa + papas (pendiente).

A.4 Conclusión de la evidencia local

Las siete capturas anteriores documentan, de forma directa y reproducible, el funcionamiento del MVP en un entorno externo distinto al de desarrollo original. El sistema se levanta en menos de tres minutos siguiendo los pasos del README, expone los cinco módulos definidos en la documentación, responde a los accesos HTTP del navegador con código 200 y presenta los datos seed correctamente. Esta evidencia cierra el criterio de la rúbrica que exige que los componentes funcionen según lo detallado en la documentación, en respuesta directa a la observación recibida en la Sumativa 2.